DEPSTAR

Devang Patel Institute of Advance Technology and Research

(A Constitute Institute of CHARUSAT)

Certificate

This is to certify that

Mr./ ~~Mrs.~~ Pal Ghori

of 3 DCE-1 Batch B *Class,*

ID. No. 25DCE030 *has satisfactorily completed*

his/ her term work in ITUE 203 (WDF) *for*

the ending in Oct 2026/2027

Date : 22/10/26

Sign. of Faculty

Head of Department

Practical - 1

Practical Title: Project Initiation,Requirement Analysis, Sitemap, Wireframe, and GitHub Setup

Problem Definition: Initiate the semester-long StudentHub portal by identifying the problem scope, user roles, key modules, navigation flow, and minimum 10 pages. Create a sitemap, low-fidelity wireframe, project folder structure, README file, and GitHub repository.

Theory

1. What is URL and parts of URL?

Answer: URL is Uniform Resource Locator , which is a unique link/address of anything like website or webpage . The following are the parts of URL :- scheme , domain , subdomain , top level domain , port , path , query string and fragment.

1. How does HTML file process in web browser?

Answer: A web browser processes an HTML file through a sequence called the Critical Rendering Path, transforming raw code into a visual webpage.

3. How will page navigation flow managed among all HTML pages?

Answer: Page navigation flow among HTML pages is managed in three primary ways:-

Hyperlink , browser history , javascript routing .

4. How will GitHub commits be maintained after each practical?

Answer: GitHub commits are typically maintained after each practical by making self-contained commits for logical milestones and pushing them to a remote repository.

Implementation

In this practical, the foundational structure of the StudentHub Portal was implemented using HTML5. The following key elements and tags were utilized:-

BasicHTML Skeleton:- Used <!DOCTYPE html>, <html>, <head>, and <body> to structure all 10 web pages.

Navigation:- Implemented a consistent global navigation bar across all pages using <nav>, unordered lists <ul>, and list items <li> with anchor tags <a> for links.

Forms & Input:- Created user authentication and contact pages
(login.html, register.html, contact.html) using the <form> tag along with various <input> types (text, email, password) and <button> elements.

Content Organization :- Used semantic tags like (<header> ,<footer> ,<main>) and container sto organize the dashboard

Sitemap Generation:- Created a sitemap.xml file using standard XML tags
like <urlset>, <url>, <loc>, and <priority> to define the structure and navigation

Evidence of Work (Screenshots)

Description: wireframe image of login page

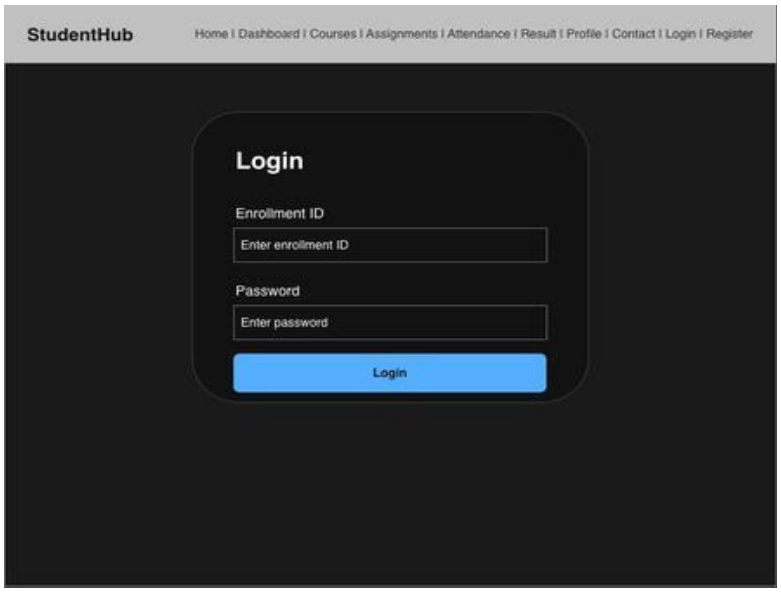


Figure 1.1

Description: wireframe image of index page

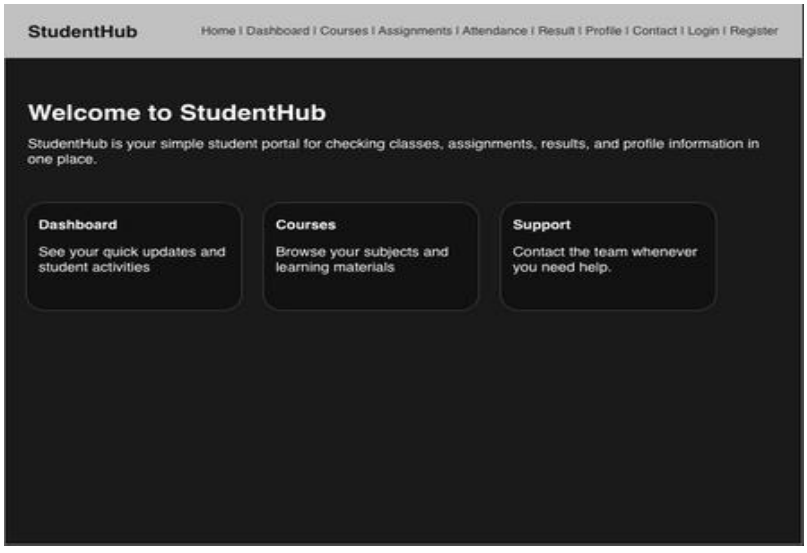


Figure 1.2

Description: wireframe image of assignment page

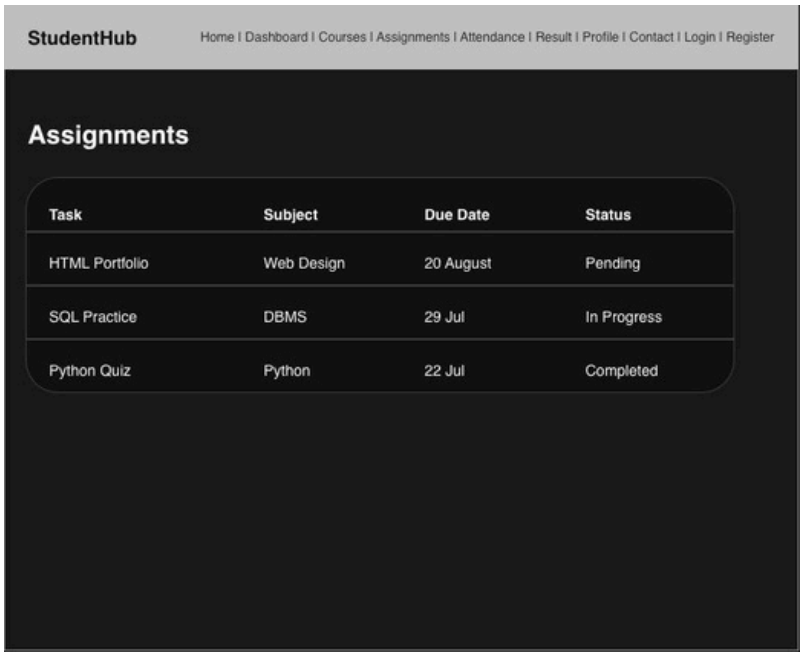


Figure 1.3

Description:wireframeimageofattendance page

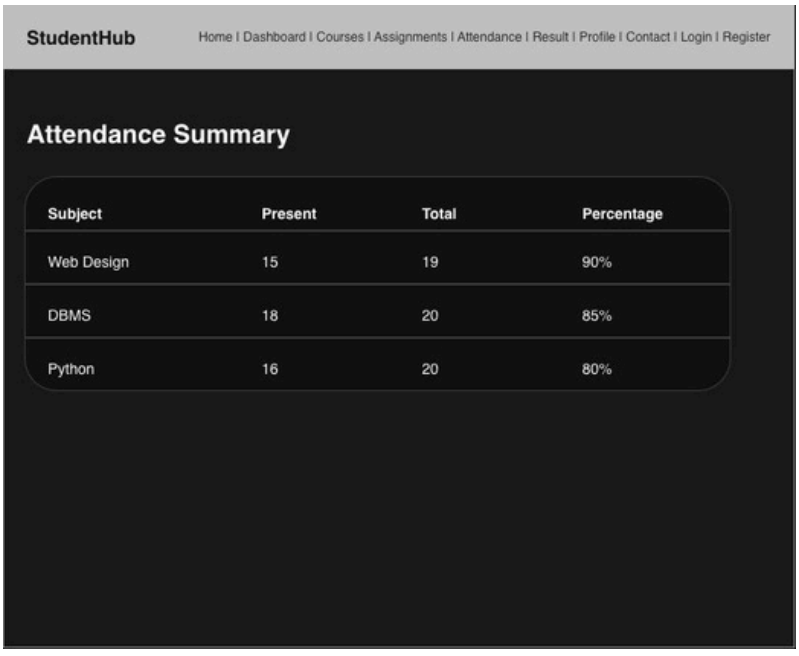


Figure 1.4

Description: wireframe image of contact page

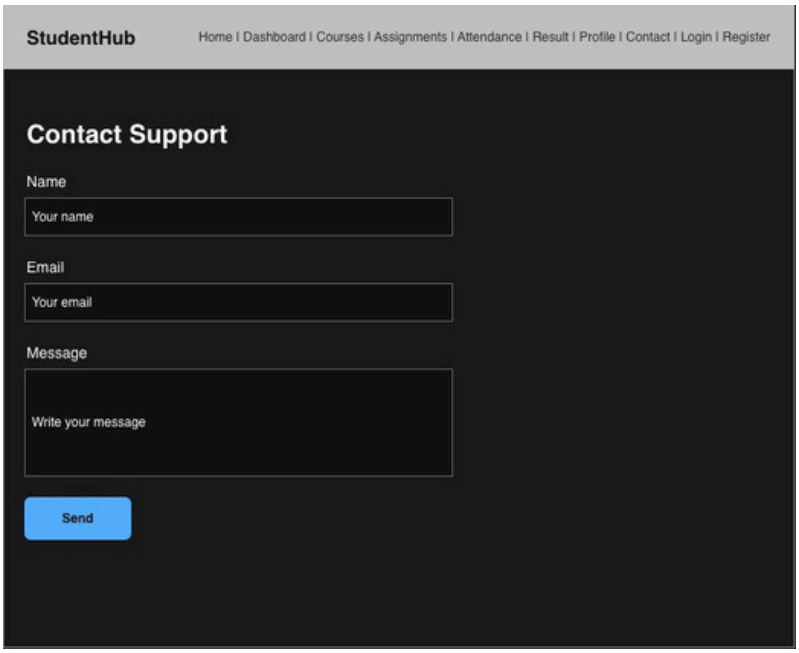


Figure 1.5

Description:wireframeimageofcoursespage

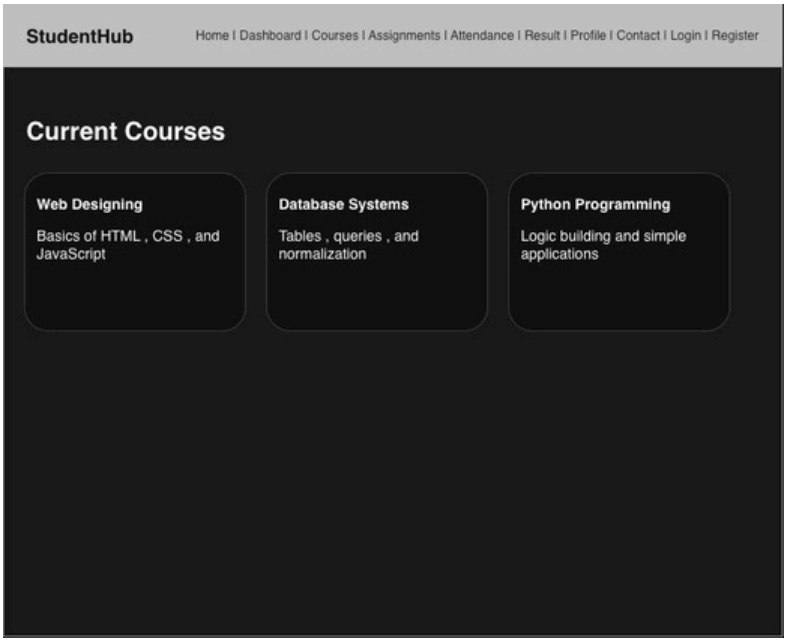


Figure 1.6

Description: wireframe image of dashboard page

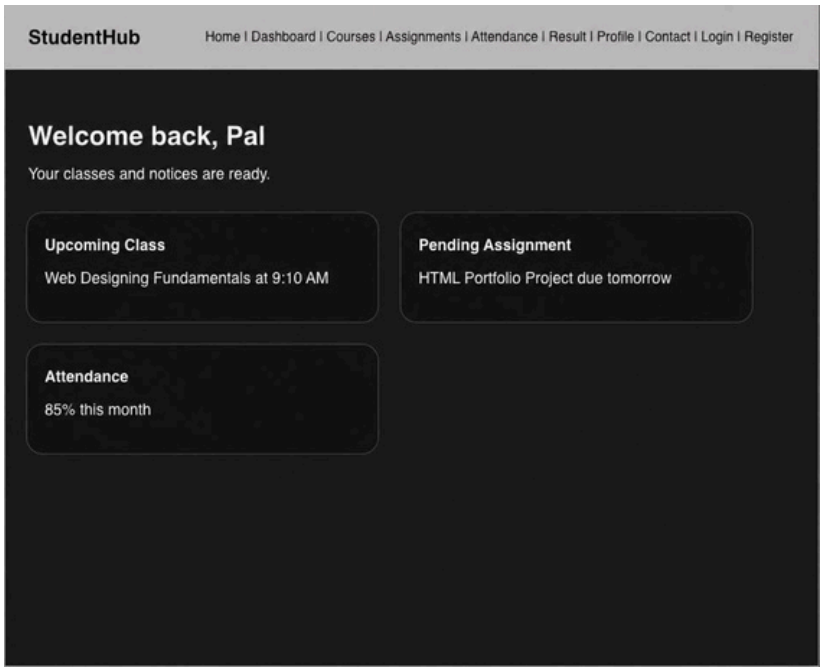


Figure 1.7

Description:wireframeimageofprofilepage

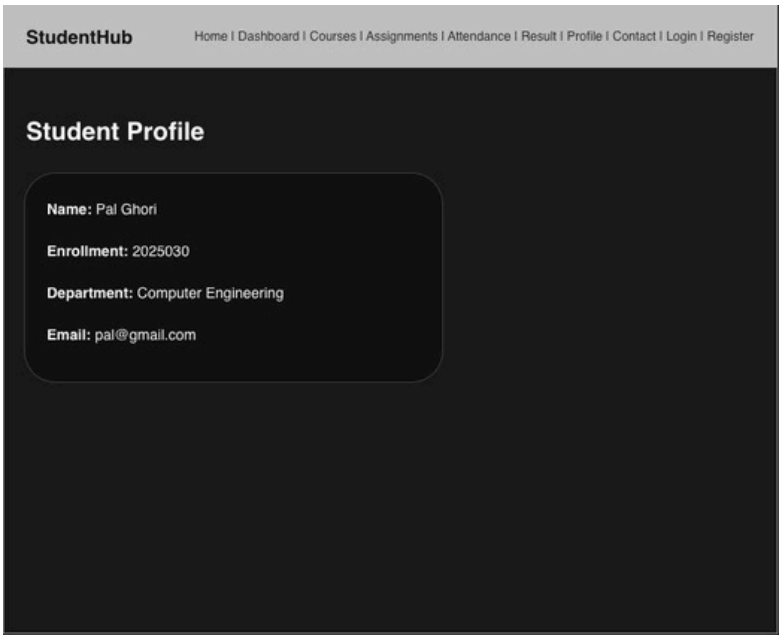


Figure 1.8

Description: wireframe image of register page

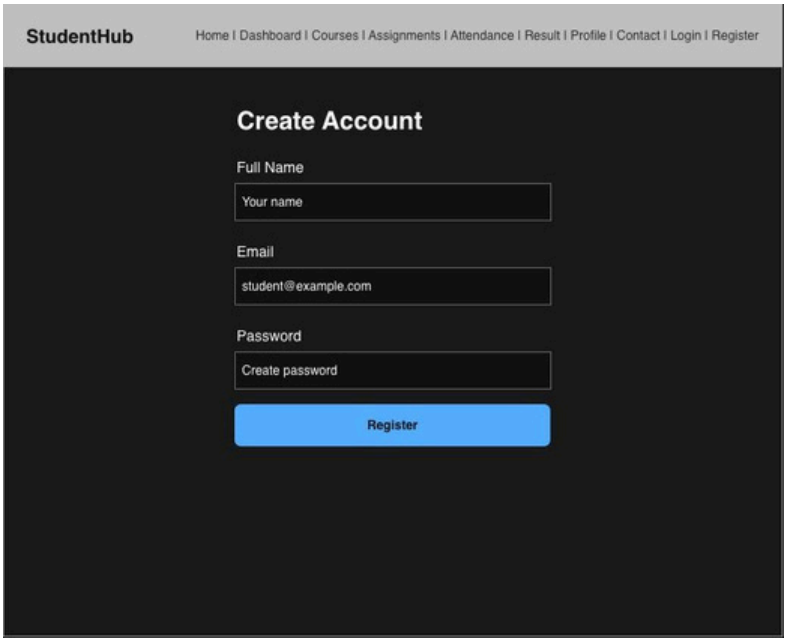


Figure 1.9

Description:wireframeimageofresultpage

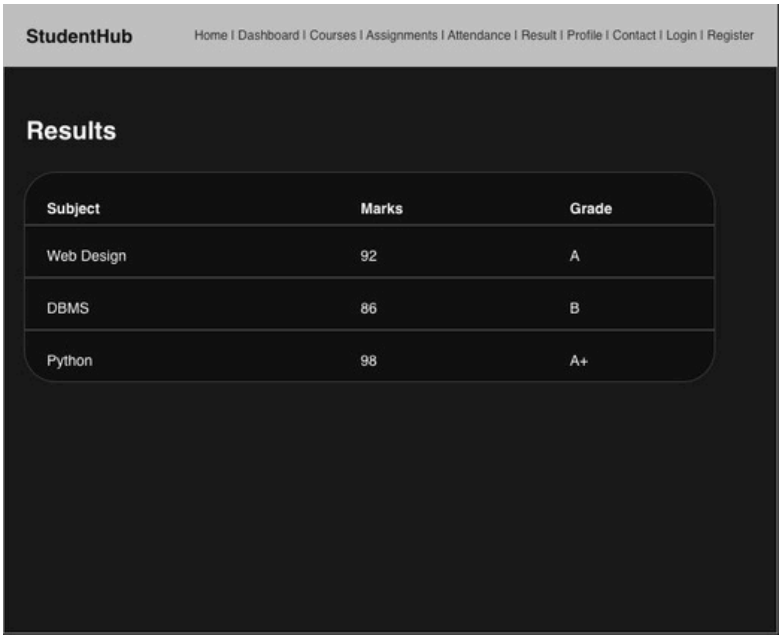


Figure 1.10

GitHub Repository Link:

https://github.com/palghori/STUDENT-HUB-PORTAL-WDF/tree/main/practical%201

Challenges Faced:

During Practical 1, the primary challenge was translating the broad idea of a StudentHub into specific, actionable requirements without overcomplicating the initial project scope. Designing the wireframes in Draw.io presented difficulties in planning how to clearly display complex data, like the dashboard summaries and course tables, on a single screen. It was also challenging to ensure that the global navigation and UI layouts remained perfectly consistent across all 10 distinct page designs during the wireframing phase. When creating the sitemap, I had to adapt to the strict syntax rules of XML, ensuring all <url>, <loc>, and <priority> tags were accurately nested and closed properly. Finally, deciding on a logical folder architecture (CSS, JS, IMAGES, PAGES) was tricky but necessary to ensure the project remains organized during future coding phases.

Learning Outcomes:

Gained hands-on experience structuring a multi-page web application using standard HTML5 tags and semantic elements.

Learned how to implement global navigation and link multiple HTML files together across different directories.

Mastered the process of organizing a clean project architecture with dedicated folders for CSS, JS, and HTML pages.

Developed an understanding of how to build structural layouts for forms, tables, and dashboards using pure HTML.

Learned how to write and format a sitemap.xml file to define the website's hierarchy for search engines.

Faculty Remarks:

Practical - 2

Practical Title: Semantic HTML5 Pages with Accessibility-Ready Structure

Problem Definition: Develop static HTML5 skeletons for at least 10 StudentHub pages such as

Home, About, Register, Login, Dashboard, Events, Profile, Contact,

Admin, FAQ, and Feedback. Use semantic tags and accessibility-friendly structure.

A. Theory

1. Are semantic tags such as header, nav, main, section, article, aside, and footer used properly?
Answer:

No, they are only partially used.

Used correctly :- <nav>, <main>, and <section> are used properly across the pages.

Not used at all :- <header>, <footer>, <article>, and <aside> are missing from the entire folder. (e.g., The <nav> is not wrapped in a <header>, and there are no <footer> tags at the bottom of any page).

2. Are form labels, headings, image alt text, and navigation links accessible ?
Answer :

Yes . Form labels are correctly bound to inputs using for and id attributes (e.g., <label for="enrollment">). Headings (<h1>, <h2>) and navigation links (<a>) are used semantically.

3. Is the page structure consistent across all pages?
Answer :

Yes . The page structure is perfectly consistent.

Every page utilizes the exact same <nav class="navbar"> block at the top, followed by a<main> container for the core content.

B.Implementation

Inthis practical, the foundational structure of the StudentHub portal was developed.

Ten separate HTML pages (such as dashboard, courses, login, and result) were coded using core HTML5 elements.

Semantic tags, complex <table> structures for data presentation, and <form> elements for user input were implemented to build the raw, unstyled framework of the application.

C. Evidence of Work (Screenshots)

Description: home page of student_hub

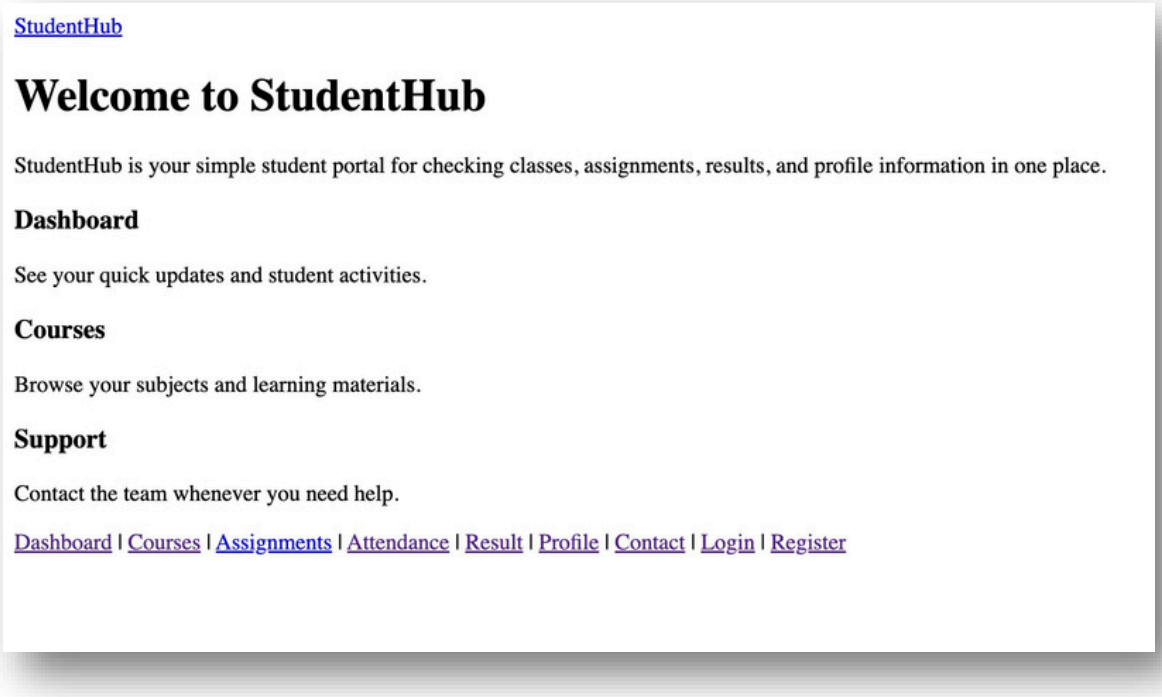


Figure 2.1

Description: dashboard page of student_hub

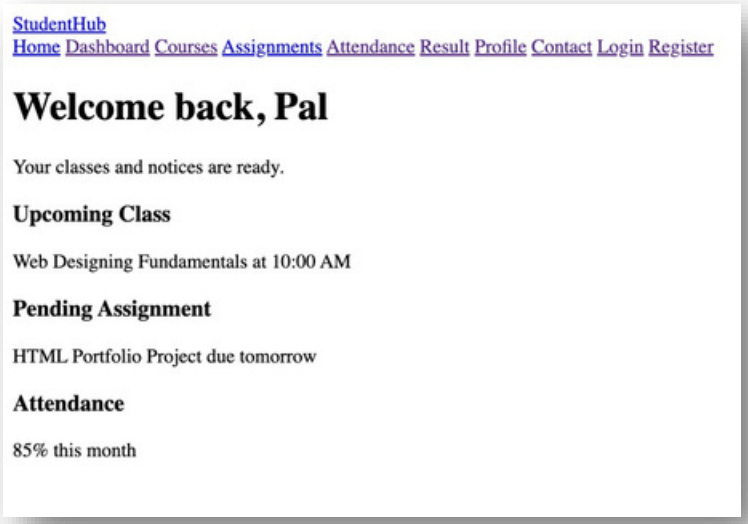


Figure 2.2

Description: courses page of student_hub

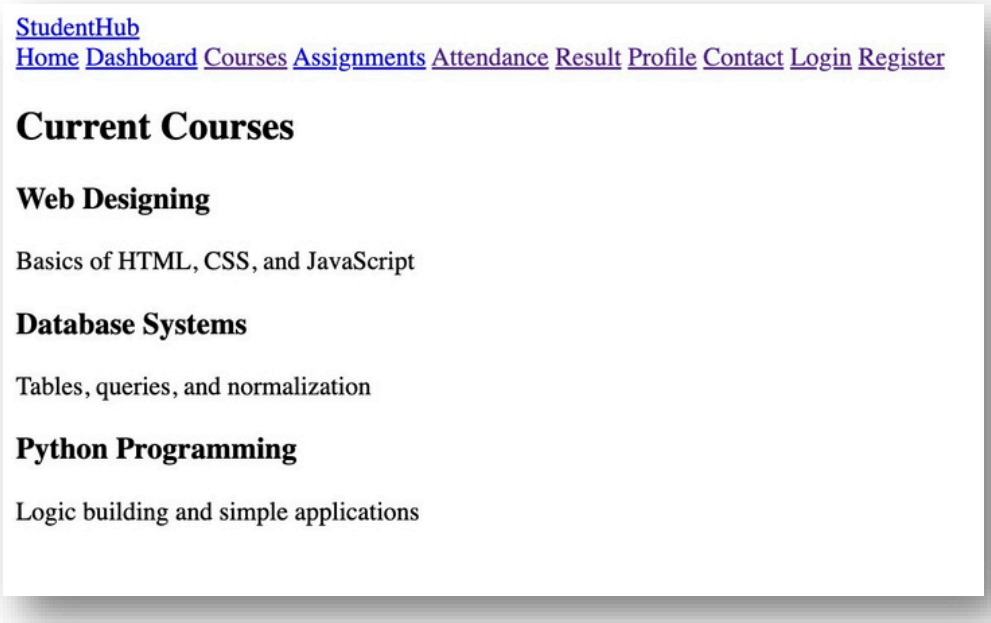


Figure2.3

Description: attendance page of student_hub

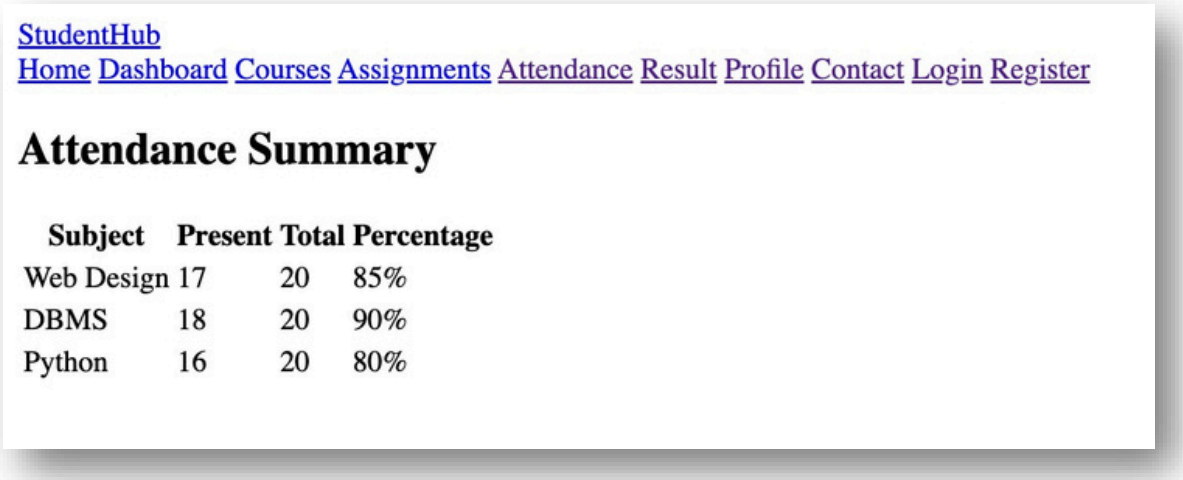


Figure2.4

Description: result page of student_hub

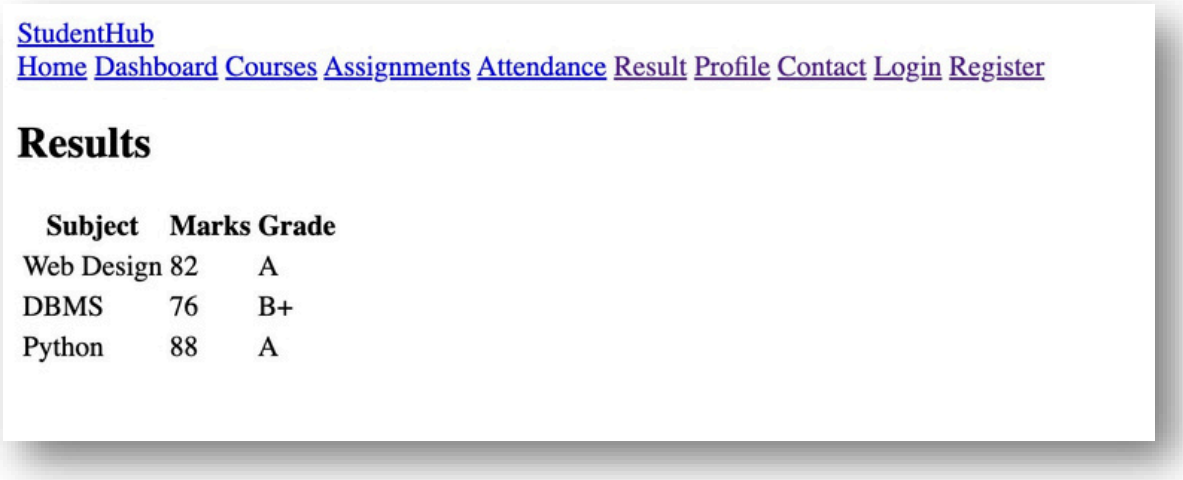


Figure2.5

Description: contact page of student_hub

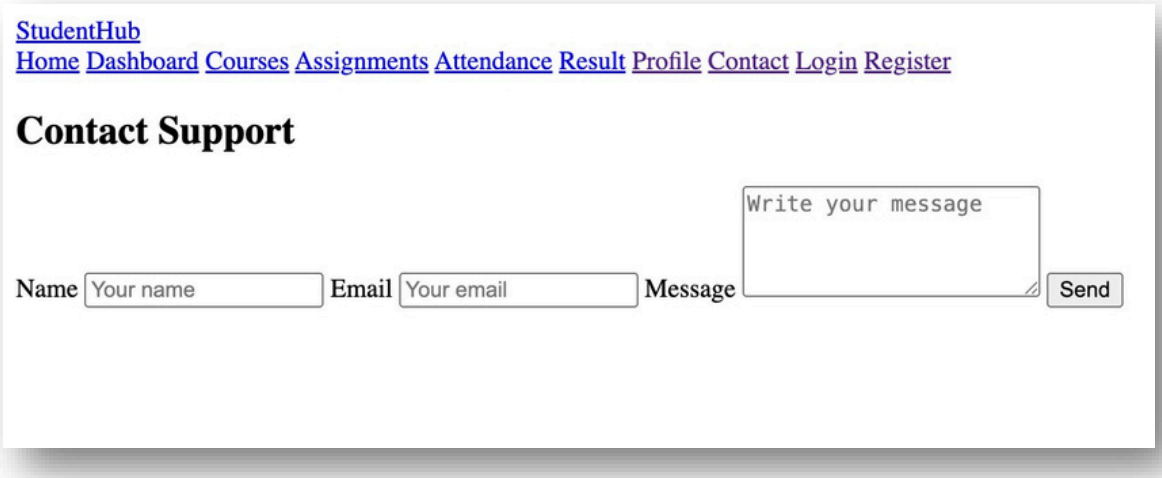


Figure2.6

Description: profile page of student_hub

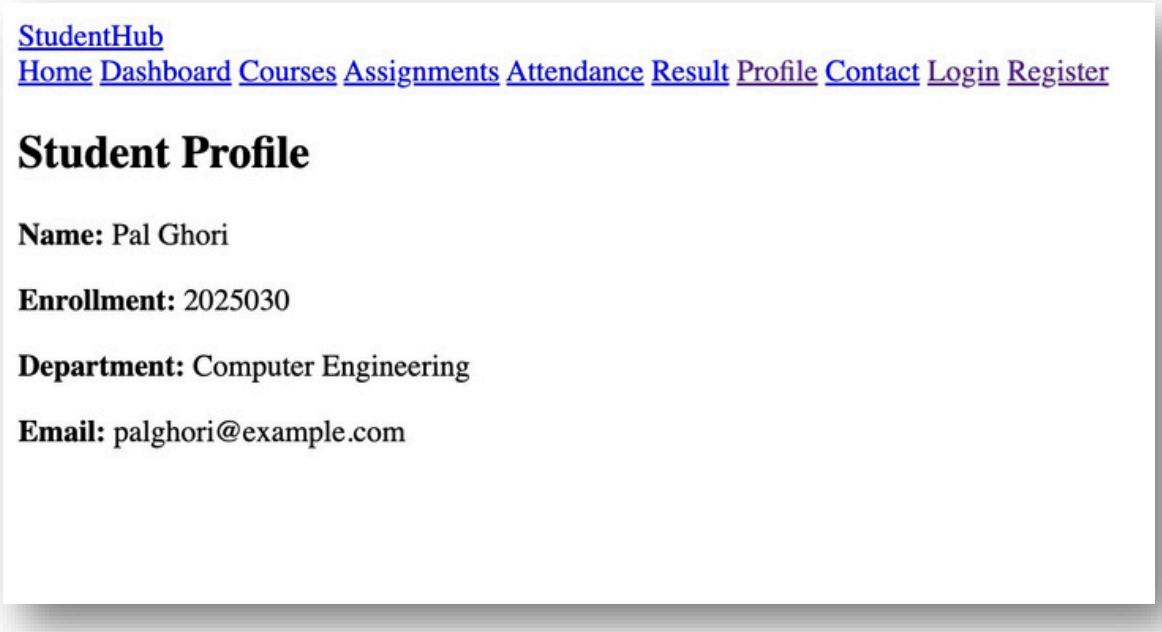


Figure 2.7

Description: assignment page of student_hub

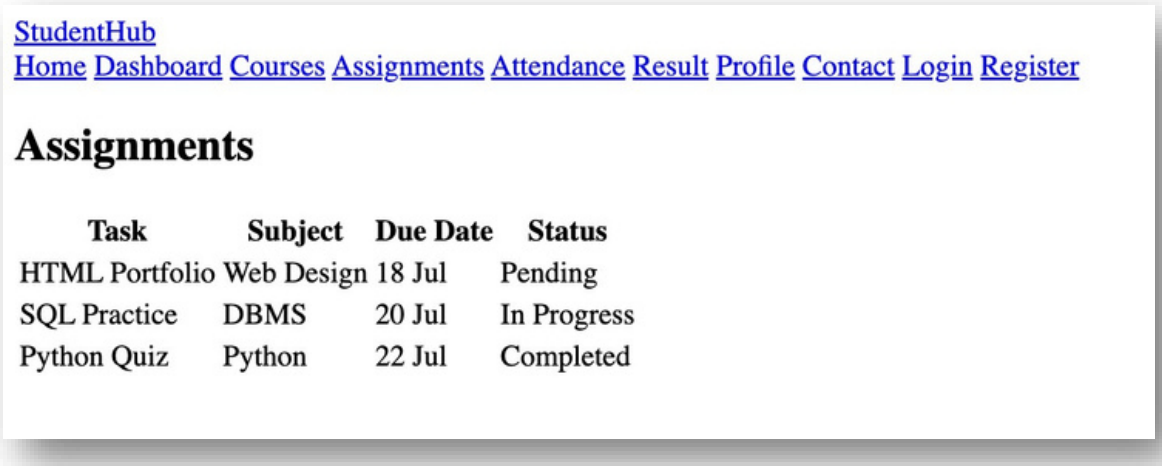


Figure2.8

Description: register page of student_hub

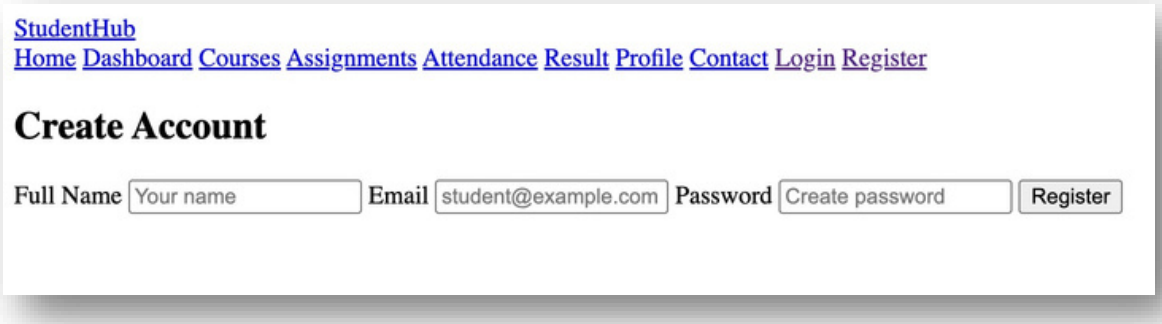


Figure 2.9

Description: login page of student_hub

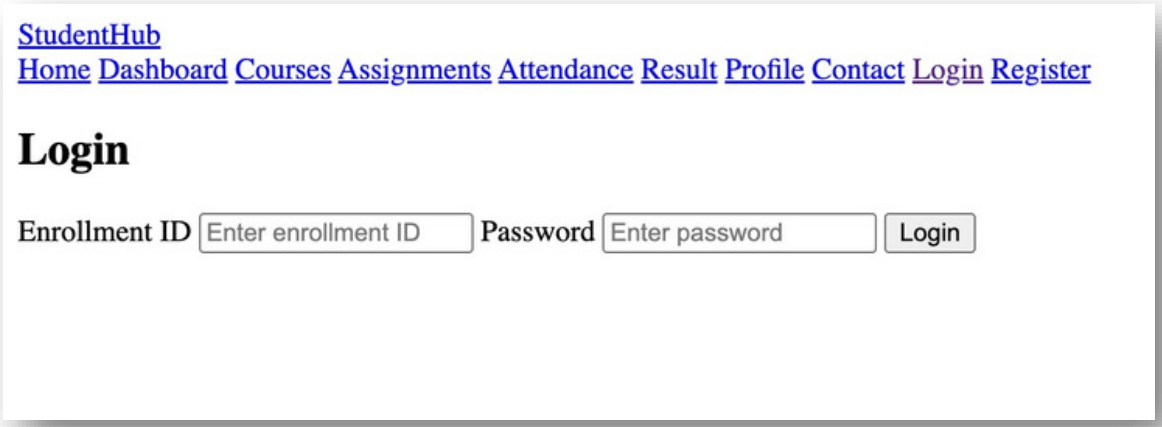


Figure2.10

D. GitHub Repository Link:

https://github.com/palghori/STUDENT-HUB-PORTAL-WDF/tree/main/practical%202

E. Challenges Faced:

During Practical 2, the main challenge was building the portal strictly using unstyled HTML elements while ensuring the content remained logical and readable.

Creating complex pages like the results and dashboard required careful nesting of <table>, <tr>, and <td> tags to present data clearly.

Another hurdle was designing the forms for login and registration, ensuring all <input> fields had the correct attributes for future backend integration.

Furthermore, manually linking 10 distinct files using <a href="filename.html"> tags meant that testing the navigation flow and preventing dead links was a tedious and error-prone process.

F.Learning Outcomes:

Mastered the creation of structured, multi-page HTML5 documents.

Learned how to build complex data tables and interactive user input forms.

Gained experience using semantic HTML to ensure clear document structure.

Understood how to manually link static web pages to form a cohesive portal.

Faculty Remarks:

Practical - 3

Practical Title: Responsive UI Design using CSS Grid, Flexbox, and Bootstrap/Tailwind

Problem Definition: Design responsive layouts for Home, About, Registration, Dashboard, and Events pages using CSS Grid and Flexbox. Optional to use CSS framework such as Bootstrap or Tailwind. Ensure mobile-first responsiveness.

A. Theory

1. How does the layout adapt for mobile, tablet, and desktop screens?

Answer :

The layout adapts fluidly. Instead of strict media queries, it uses responsive CSS properties like flex-wrap: wrap and min-width: 180px (seen in .feature-box). This allows elements to automatically stack on mobile/tablet screens and sit side-by-side on desktop screens.

2. Which layout technique is used and why?

Answer :

CSS Flexbox (display: flex;) is the primary layout technique used (specifically for .features and form containers). It is used because it provides a modern, efficient way to align elements, distribute space, and handle dynamic wrapping without relying on complex and outdated float rules.

3. Are typography, spacing, colors, and contrast readable?

Answer :

Yes.Typography uses clean, modern fonts (Segoe UI, Arial). Spacing is consistent (e.g.,padding: 24px, gap: 15px). The color palette uses a warm, earthy theme (browns on beige),providing excellent contrast and high readability between the dark text (#3a2a1a) andlightbackgrounds (#fffdfa).

4.Isthe framework used consistently without unnecessary inline CSS?

Answer :

Yes,perfectly**.** Pure CSS3 is used consistently via a single external stylesheet (../CSS/script.css). A scan of all HTML files in Practical 3 confirms there is ***zero*** inline CSS(nostyle="..." attributes used anywhere), ensuring a clean separation of structure anddesign.

B. Implementation

This practical focused on UI design and project organization.

The flat file structure was reorganized into a professional architecture using CSS, JS, and PAGES directories.

A central script.css stylesheet was created and linked to all pages, utilizing advanced CSS techniques like linear gradients, box shadows, Flexbox layouts, and hover effects to transform the raw HTML into a polished, modern interface.

C. Evidence of Work (Screenshots)

Description: home page of student hub portal

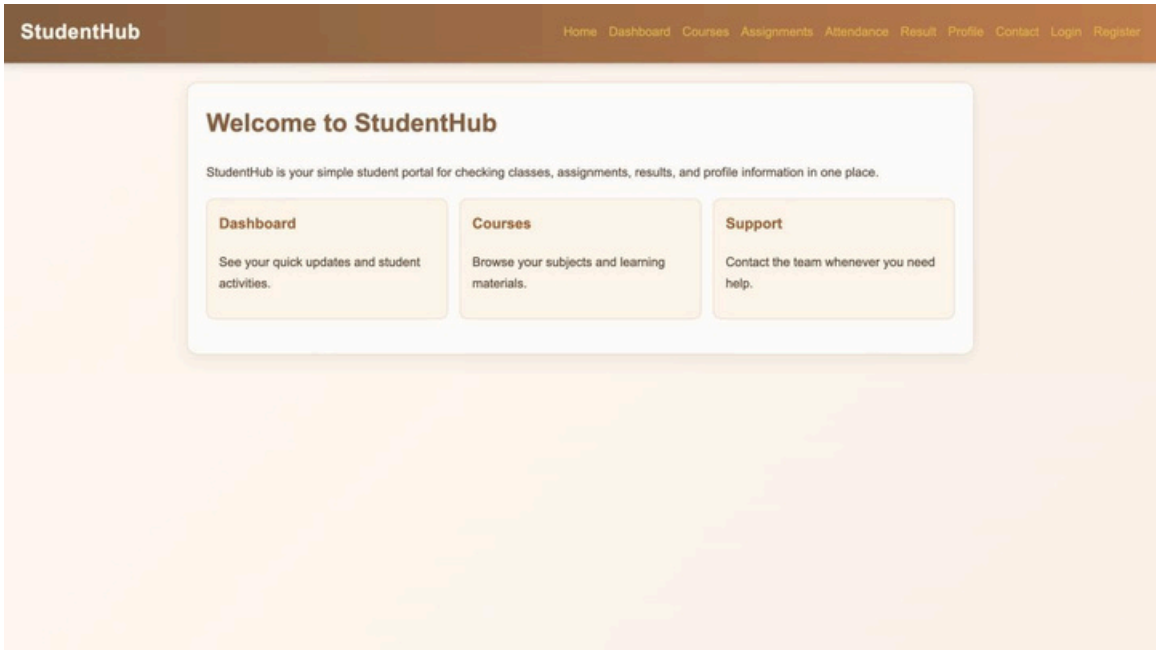


Figure 3.1

Description: login page

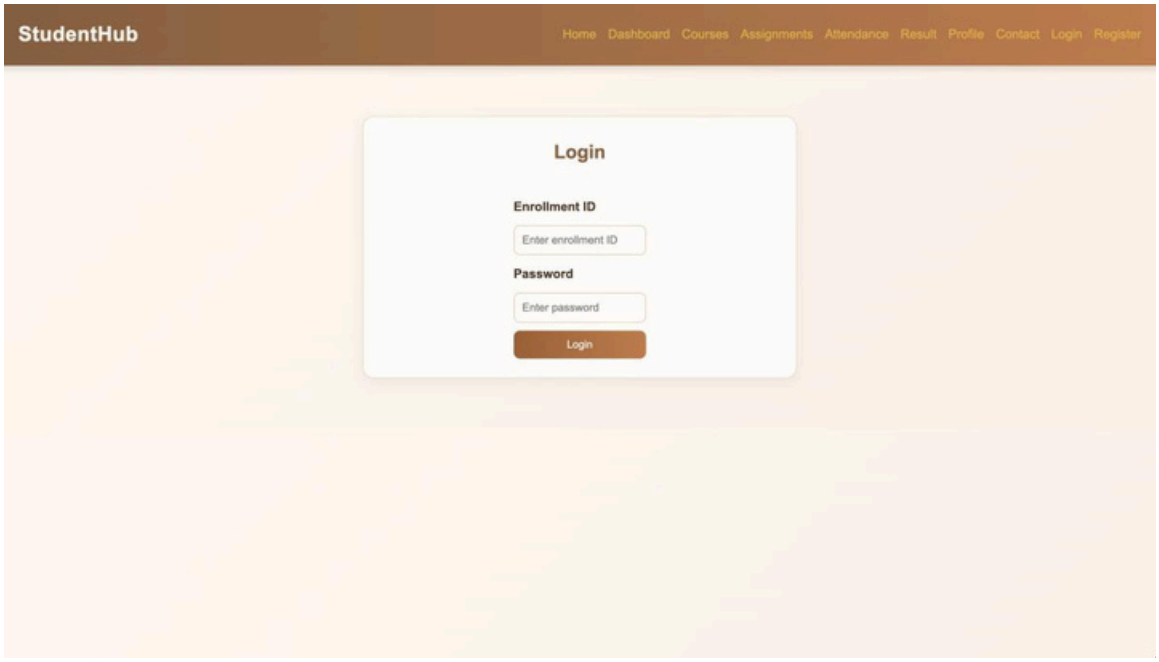


Figure 3.2

Description: register page

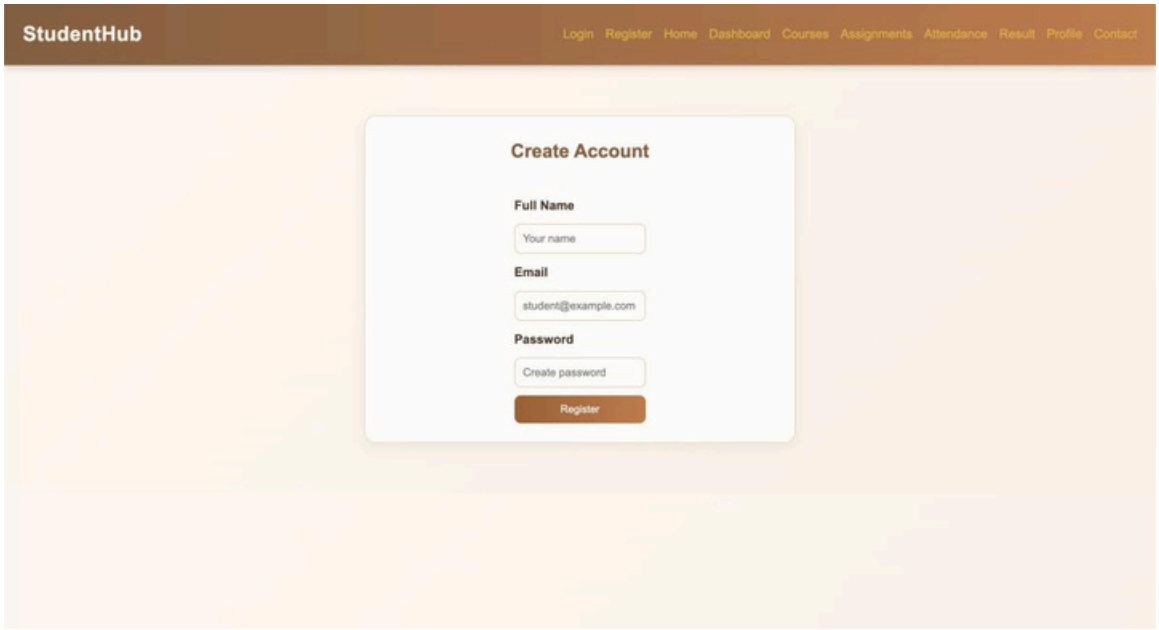


Figure 3.3

Description: profile page

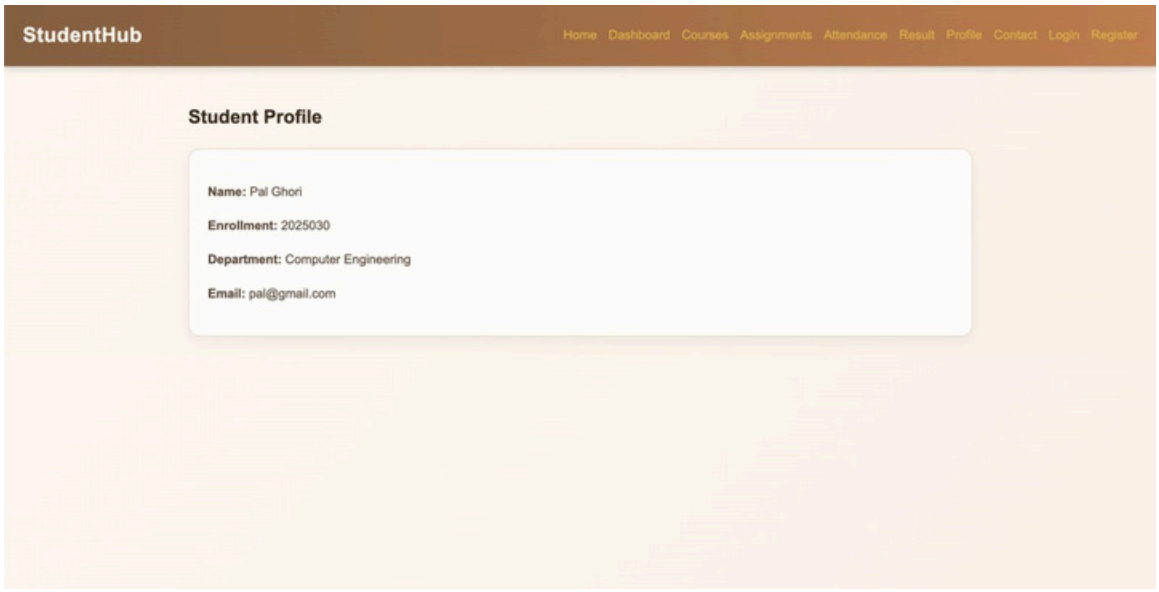


Figure 3.4

Description: contact page

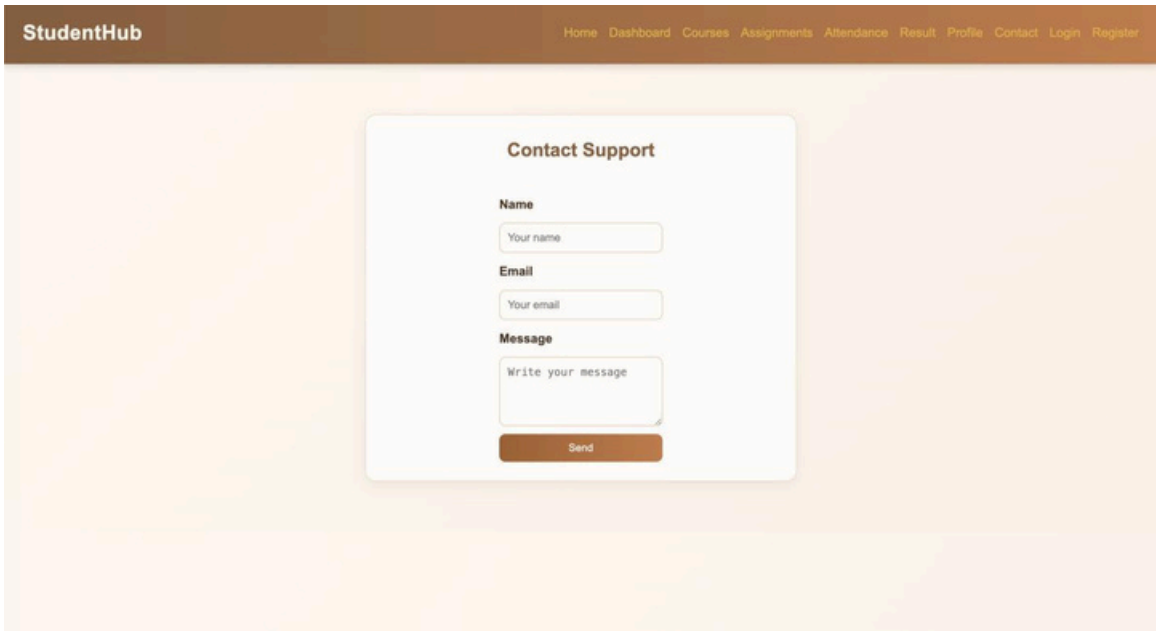


Figure 3.5

Description: result page

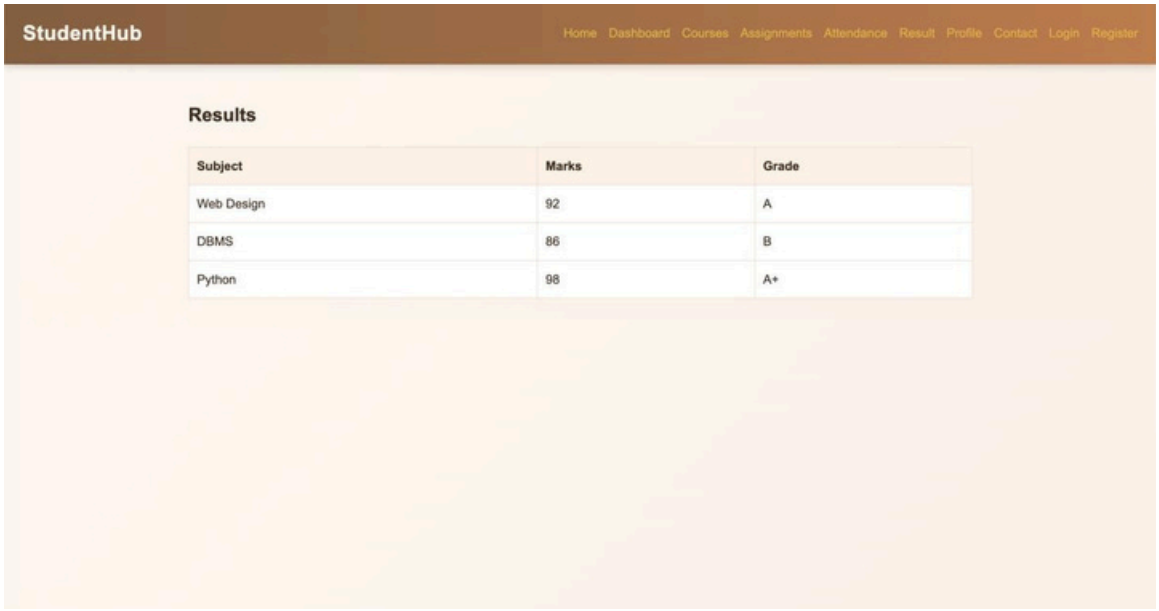


Figure 3.6

Description: attendance page

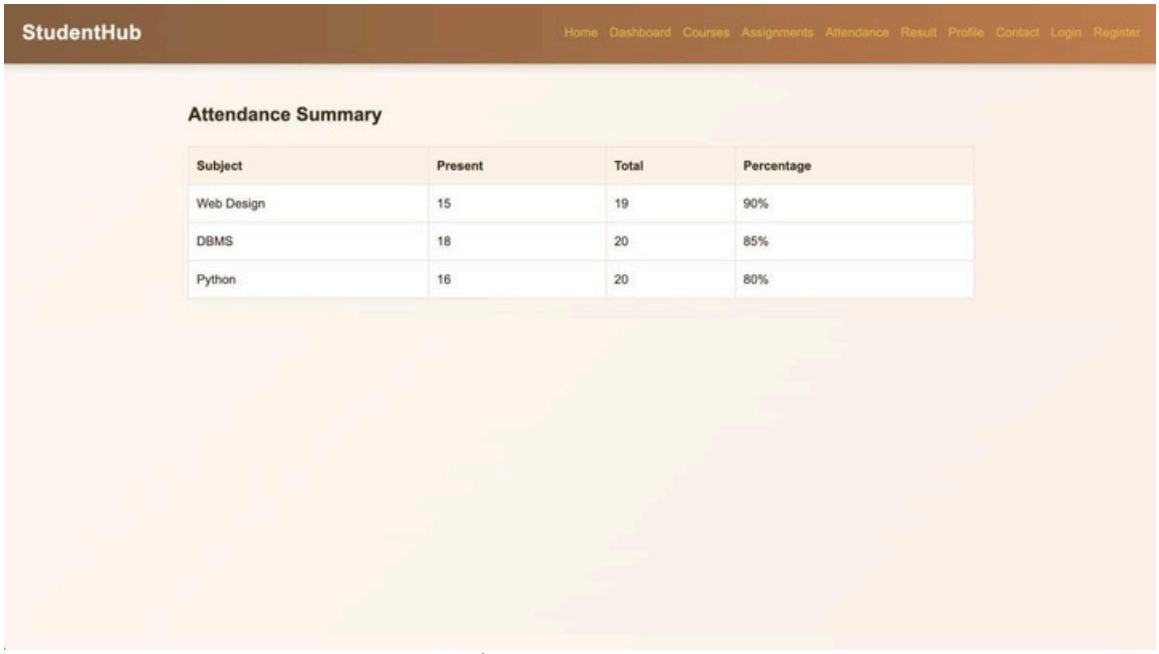


Figure 3.7

Description: assignment page

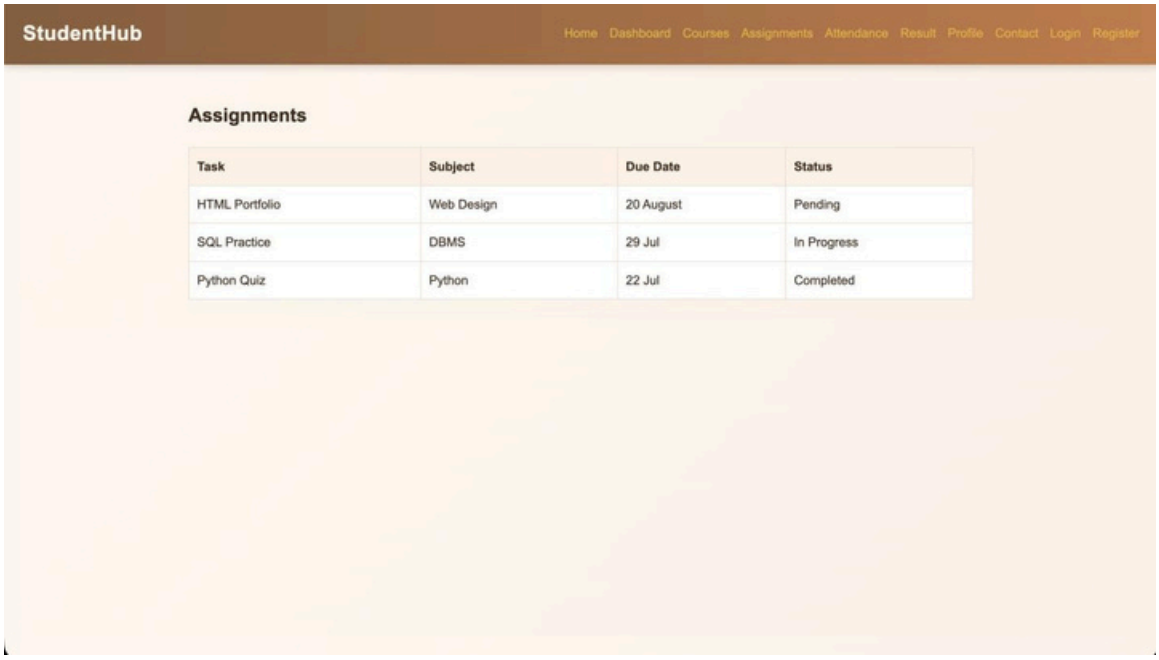


Figure 3.8

Description: courses page

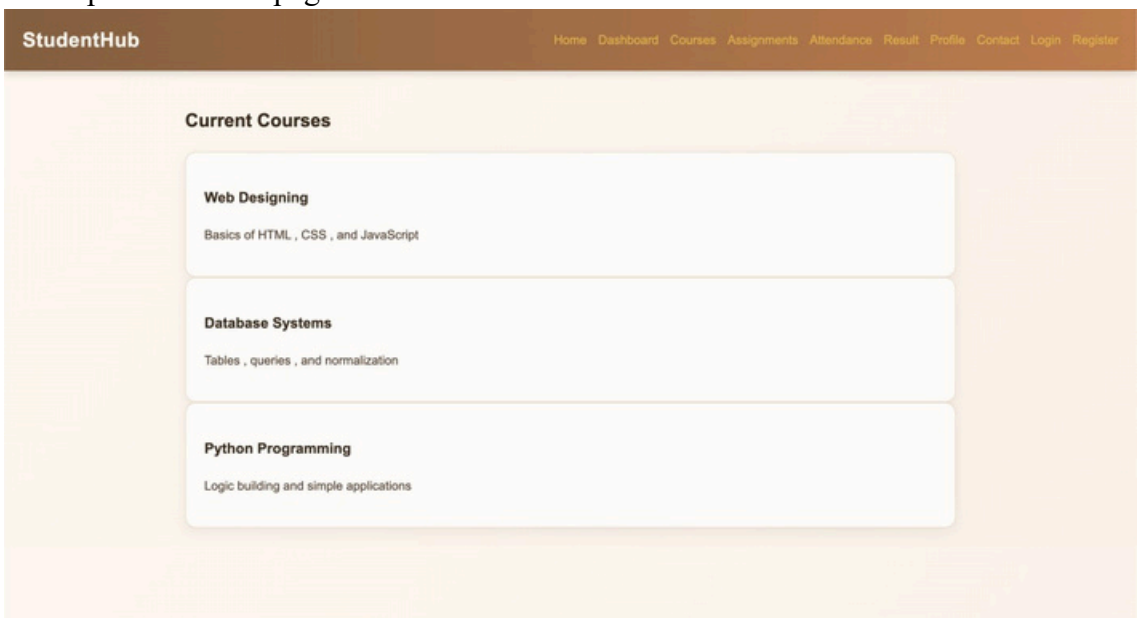


Figure 3.9

Description: dashboard page

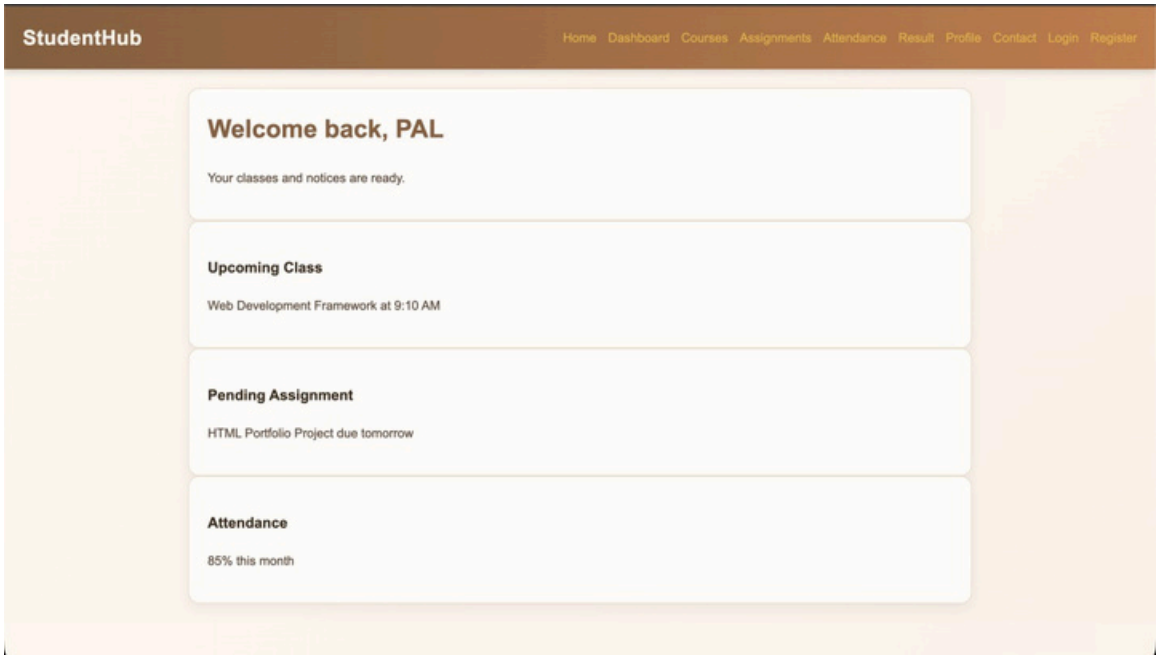


Figure 3.10

D. GitHub Repository Link:

https://github.com/palghori/STUDENT-HUB-PORTAL-WDF/tree/main/practical%203

E. Challenges Faced:

In Practical 3, moving the raw HTML into a structured directory architecture (PAGES, CSS, JS) broke all existing file links, which was challenging to fix by updating every ../ relative path.

When writing script.css, applying Flexbox to the feature boxes and form containers was tricky; getting the items to align and wrap correctly required constant tweaking.

Additionally, implementing consistent styling for tables and navigation bars across all pages meant I had to rely heavily on CSS classes and global rules, which was a significant shift in thinking.

Finally, debugging conflicting CSS rules where styles accidentally overrode each other took considerable time.

F.Learning Outcomes:

Learned to structure a web project professionally using dedicated directories.

Gained experience applying aesthetic properties like gradients, shadows.

Learned how to resolve relative path issues when linking external stylesheets.

Understood the importance of a single global stylesheet for maintaining UI

consistency.

Faculty Remarks:

Practical - 4

JavaScript DOM Manipulation, Event Handling, and UI Interactivity

Problem Definition: Add dynamic UI components such as collapsible FAQ, modal popup, image/content slider, notification banner, hamburger menu, and light/dark theme switcher using JavaScript.

A. Theory

1. How are DOM elements selected and modified?

Answer :

They are selected using document.getElementById('enrollment') and document.body. They are modified by retrieving input values (.value) and updating CSS classes (classList.toggle() and classList.add()).

2. Are event listeners attached correctly?

Answer :

They are functional, but it uses a mix of standard listeners (document.addEventListener('DOMContentLoaded')) and inline HTML handlers (onsubmit="validateAuth(...)", onclick="toggleDarkMode()"). While it works, standardizing on addEventListener is generally best practice.

3. Is localStorage used for remembering theme choice?

Answer :

Yes. The script uses localStorage.setItem('darkMode', ...) to save the choice and localStorage.getItem('darkMode') on page load to reapply it.

4. Does interactivity improve usability without breaking accessibility?

Answer :

It improves usability by offering dark mode and immediate form validation. However, accessibility could be better; using alert() for form errors isn't ideal for screen readers, and replacing it with semantic, inline error messages would improve accessibility.

B. Implementation

Client-Side Validation :- Added JavaScript to validate the Enrollment ID format on the login/register forms using Regular Expressions (Regex)
and event.preventDefault() to stop invalid submissions.

Theme Toggling :- Created a "Dark Mode" toggle that dynamically updates the page theme by adding/removing CSS classes via DOM manipulation (classList.toggle).

State Persistence :- Integrated browser localStorage to save the user's dark mode preference, ensuring the theme remains consistent even after refreshing or navigating across different pages.

C. Evidence of Work (Screenshots)

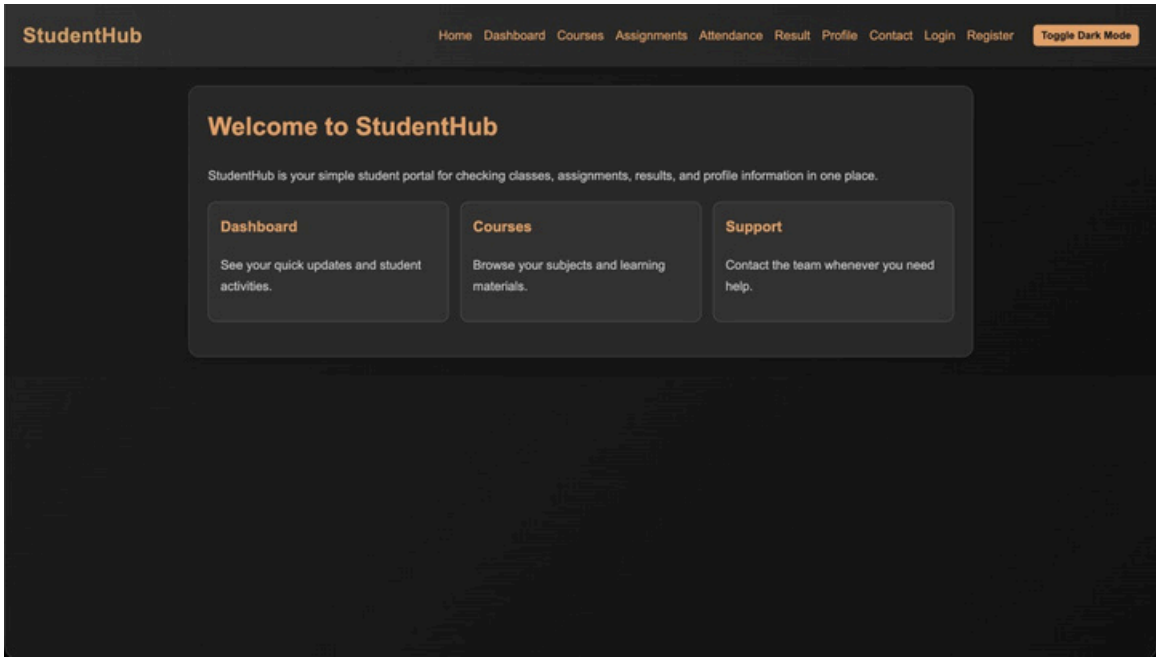


Figure 4.1 :- DARK MODE

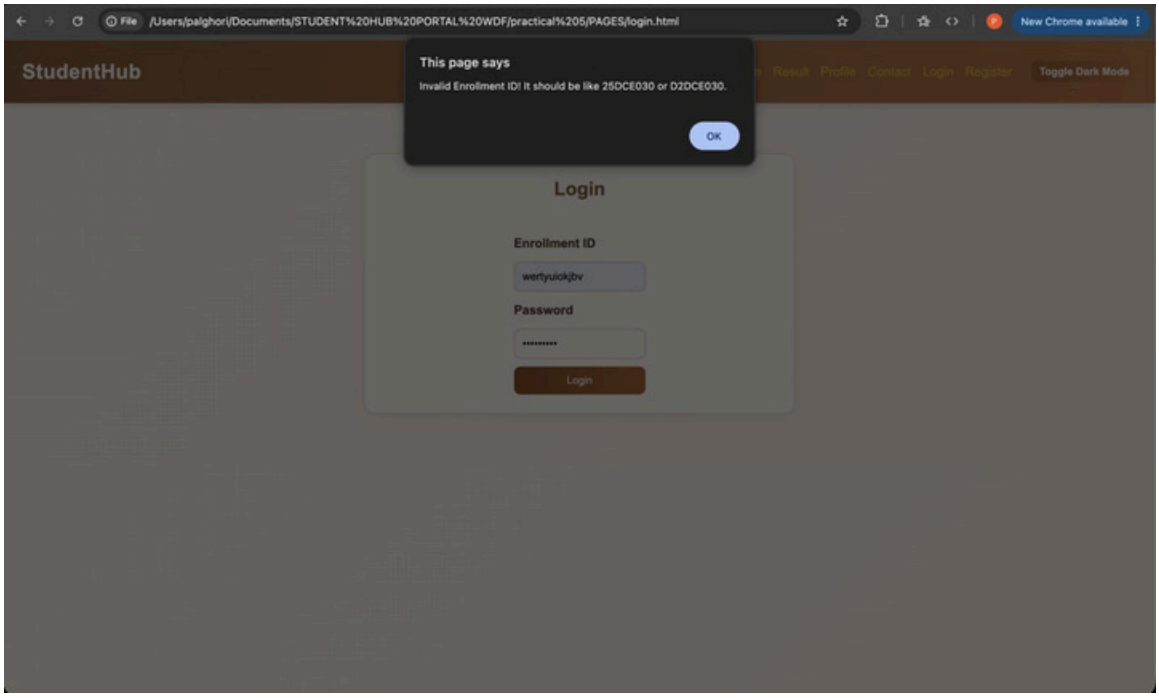


Figure 4.2 :- Regex in Login Form

D. GitHub Repository Link:

https://github.com/palghori/STUDENT-HUB-PORTAL-WDF/tree/main/practical%204

E. Challenges Faced:

Regex Pattern Matching : Constructing the exact Regular Expression (/^\d{2}[a-zA-Z]{3}\d{3}$/) to strictly enforce the specific enrollment ID format without accidentally rejecting valid inputs.

Theme State Reset : Initially, the dark mode would reset when navigating to a new page or refreshing. This required identifying localStorage as the solution to persist the state.

DOM Timing Issues : Ensuring the JavaScript for applying the saved theme only ran after the HTML had fully loaded to prevent errors, which was resolved using the DOMContentLoaded event listener.

F. Learning Outcomes:

DOM Manipulation: Gained hands-on experience selecting HTML elements (getElementById), reading input values (.value), and dynamically altering CSS classes.

Client-Side Security/Validation: Learned how to intercept form submissions and validate user input on the frontend before interacting with a backend.

Browser Storage: Understood how to utilize Web Storage APIs (localStorage) to persistently store user preferences across sessions.

Event Handling: Developed an understanding of how to listen to and manage browser events (like onsubmit, onclick, and DOMContentLoaded).

Faculty Remarks:

Practical - 5

Registration Form with Frontend Validation and User-Friendly Error Handling

Create a student registration form with HTML5 input types and JavaScript validation for name, email, mobile number, password, confirm password, course, year, gender, and terms acceptance. Use Regular Expression for validations.

A. Theory

1. Are correct input types and attributes used?

Answer :

Yes. The forms utilize HTML5 input types appropriately, such as type="email" for email validation, type="password" for masking characters, and the required attribute to leverage built-in browser constraints.

2. Are validation errors displayed near the relevant fields?

Answer :

No. Validation errors are handled via browser popups using alert("Invalid Name! ..."). They are not dynamically rendered into the HTML DOM next to the specific fields that caused the error.

3. Is password strength checked?

Answer :

No. The JavaScript only checks if the password matches the confirmPassword. There are no Regular Expressions enforcing complexity rules (like requiring numbers, uppercase letters, or special characters).

4. Is the form accessible using keyboard and screen-reader-friendly labels?

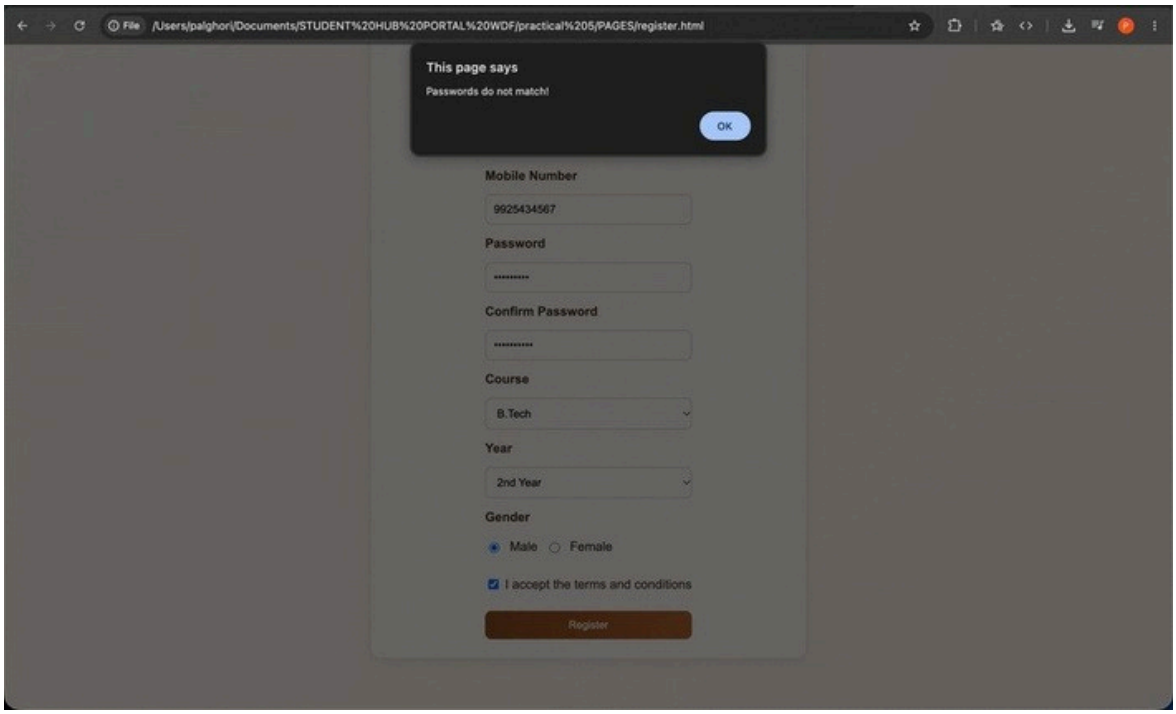
Answer :

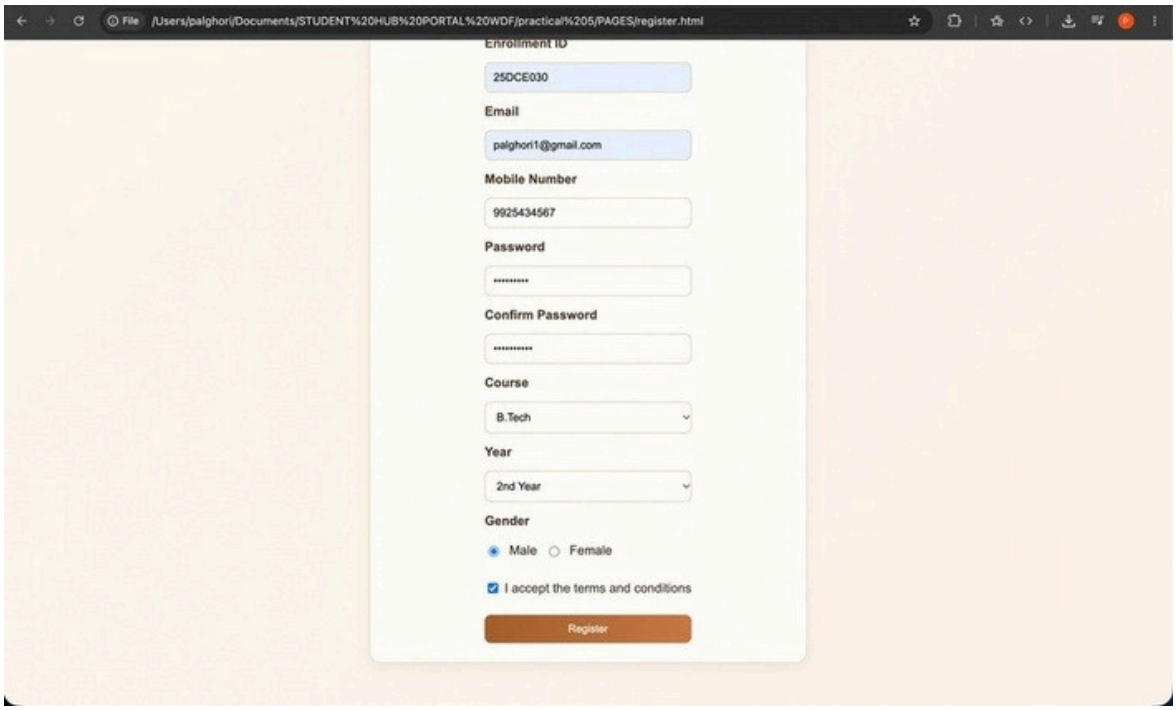
Mostly yes. The form uses <label> tags, making it somewhat screen-reader friendly, and inputs are easily navigable via the Tab key. However, not all labels use the for attribute (e.g., <label>Full Name</label>). To be strictly accessible, every label should have a for="id_of_input" attribute to explicitly link them.

B. Implementation

Expanded the JavaScript to handle comprehensive Client-Side form validation. Implemented complex Regular Expressions to validate multiple formats of Enrollment IDs (e.g., standard vs. D2D formats), strictly alphabetical Names, and exactly 10-digit Mobile numbers. Also added logic to ensure password and confirmPassword match.

C. Evidence of Work (Screenshots)




File
/Users/palghori/Documents/STUDENT%20HUB%20PORTAL%20WDF/practical%205/PAGES/register.html
Enrollment ID
25DCE030
Email
palghori1@gmail.com
Mobile Number
9925434567
Password
Confirm Password
Course
B.Tech
Year
2nd Year
Gender
Male
Female
I accept the terms and conditions
Register

D. GitHub Repository Link:

https://github.com/palghori/STUDENT-HUB-PORTAL-WDF/tree/main/practical%205

E. Challenges Faced:

- Writing and debugging robust Regular Expressions (Regex) that don't accidentally block valid data or allow incorrect formats (like /(^\d{2}[a-zA-Z]{3}\d{3}$)|(^D2D[a-zA-Z]{2,3}\d{3}$)/i).
- Capturing form submission events and safely halting them (event.preventDefault()) so that the user receives an alert() error instead of the page reloading and losing their typed data.

F. Learning Outcomes:

Gained a deep understanding of Regex pattern matching, securing forms on the frontend, and improving User Experience (UX) by providing immediate feedback on invalid inputs before the server is ever contacted.

Faculty Remarks:

Practical - 6

Rendering External JSON Data using Fetch API, Search and Filter

Display event lists, student profiles, notices, or FAQs from external JSON files using Fetch API. Implement dynamic rendering, search, filter, sorting, and pagination.

A. Theory

1. How is JSON fetched, parsed, and rendered?

Answer :

It uses the modern fetch("../DATA/events.json") API. It is parsed by returning response.json() in the first .then() block. To render, it loops through the parsed array using .forEach(), creates new div elements with document.createElement("div"), populates their innerHTML with template literals, and appends them to the #eventsList container.

2. Which array methods are used for search, filter, sort, and pagination?

- Search & Filter: The .filter() method is used to return a new array containing only events that match the text input and selected dropdown category.
- Sort: The .sort() method is used to reorder dates by comparing their timestamp values (getTime()).
- Pagination: The .slice() method is used to extract a specific chunk of the array (e.g., index 0 to 5) to display only the items for the currentPage.

3. How are loading and error states handled?

Answer :

A loading state is hardcoded into the HTML initially (Loading events...). If the fetch() request fails (network error or missing file), a .catch() block catches the error and updates the DOM to display a red error message: "Failed to load events. Please try again later."

4. How is modularity maintained in JavaScript files?

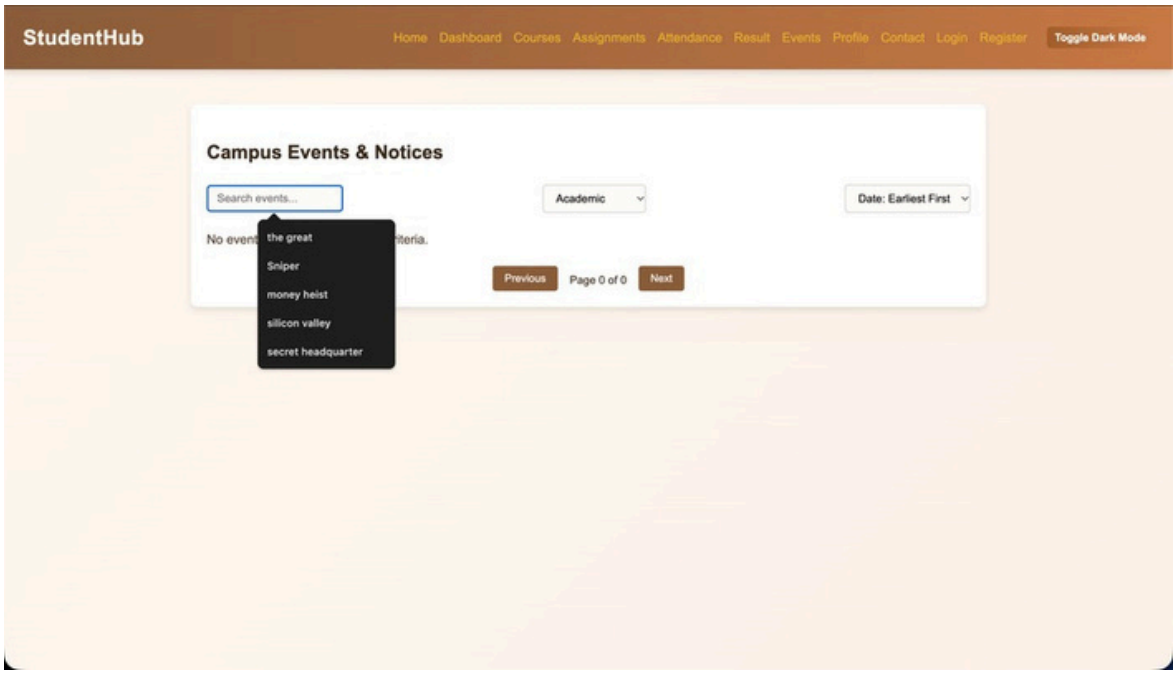
Answer :

Modularity is achieved by separating the heavy event-fetching logic into a completely separate file (JS/events.js), keeping it isolated from the global portal functions (like Dark Mode) which remain in java.js.

B. Implementation

Expanded the portal by adding a new events.html page and updating the navigation menus across all existing pages to include it. Furthermore, a Pagination UI component was implemented in script.css using CSS Flexbox to allow users to navigate through long lists (like events or courses).

C. Evidence of Work (Screenshots)



D. GitHub Repository Link:

https://github.com/palghori/STUDENT-HUB-PORTAL-WDF/tree/main/practical%206

E. Challenges Faced:

- Keeping the navigation bar synchronized across 10+ separate HTML files without a backend framework (like React or PHP includes).
- Getting the pagination buttons (Previous/Next) to align perfectly and handling their visual states (e.g., styling :disabled buttons so they look unclickable).

F. Learning Outcomes:

Improved CSS Layout skills (specifically Flexbox for the pagination controls), understanding UI component states, and experiencing firsthand the scaling limitations of purely static HTML files.

Faculty Remarks:

Practical - 7

PHP Form Processing with Server-Side Validation and CSV/JSON File Storage

Process submitted registration/contact form data using PHP. Validate and sanitize inputs on the server side and store records in CSV or JSON file format. Display success/error messages.

A. Theory

1. Is the form submitted using POST?

Answer :

Yes. Both the register.html and contact.html forms were updated to include method="POST" to securely send data to the PHP scripts without exposing data in the URL.

2. Are inputs validated and sanitized server side?

Answer :

Yes. filter_var($email, FILTER_VALIDATE_EMAIL) is used to validate email fields. Text inputs are thoroughly sanitized to prevent Cross-Site Scripting (XSS) by combining trim(), strip_tags(), and htmlspecialchars().

3. Is file writing handled safely?

Answer :

Yes. The script first checks if the DATA directory exists and creates it if it doesn't. When writing to the CSV, it uses fopen($file_path, 'a') to append data safely without overwriting previous records. When writing to JSON, it correctly retrieves the existing data, decodes it, pushes the new record, and encodes it back using file_put_contents.

4. Are success and error responses displayed clearly?

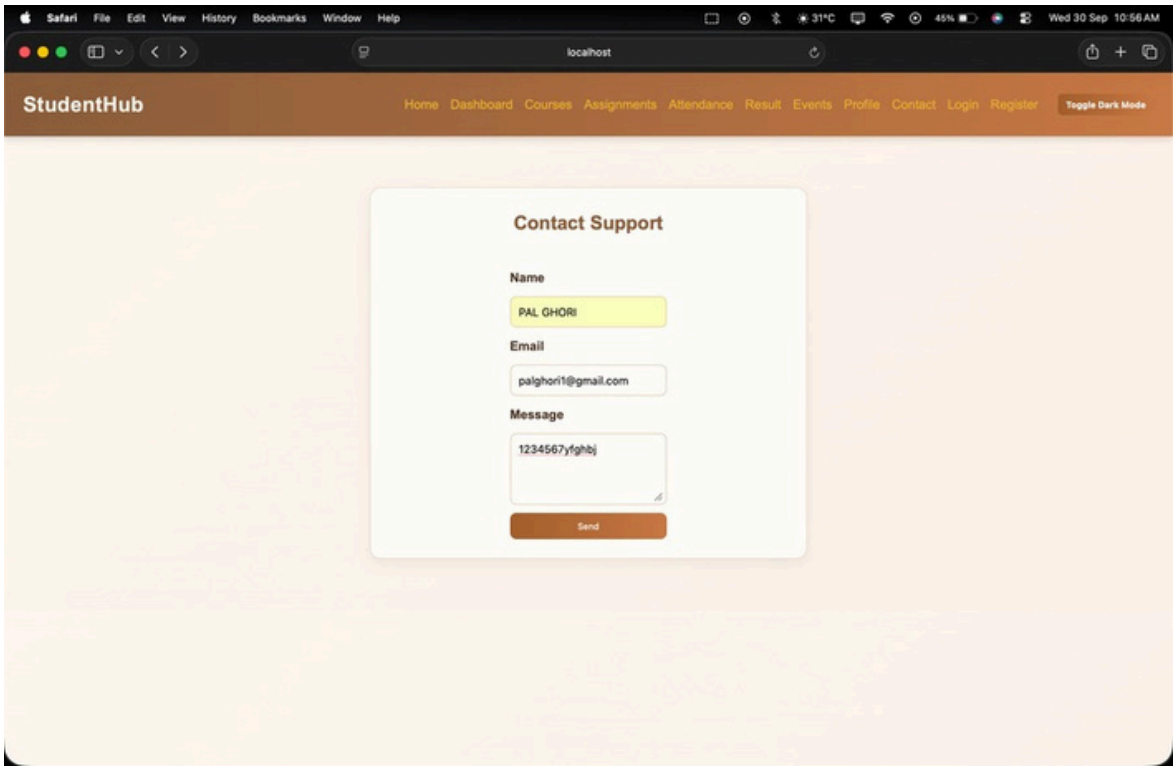
Answer :

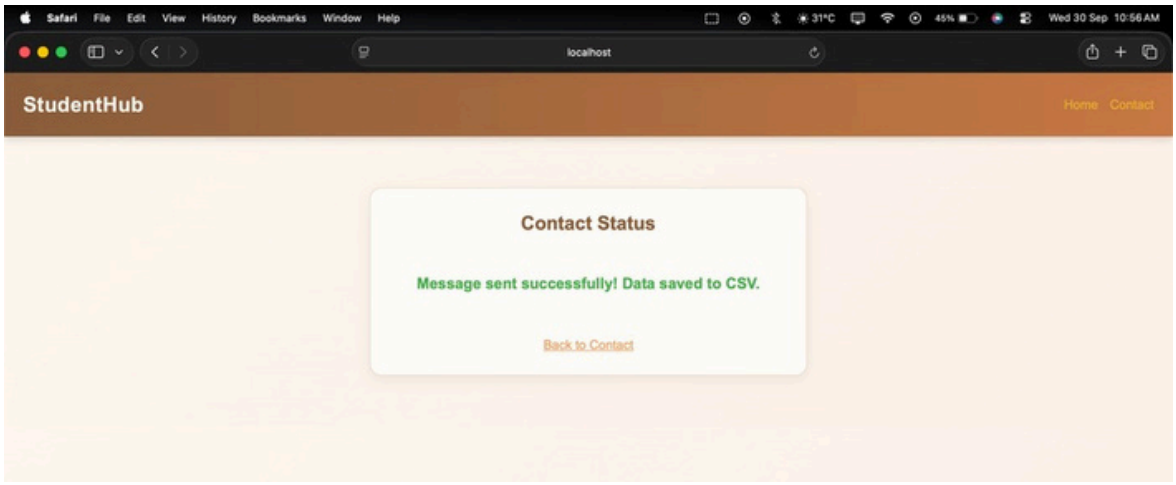
Yes. After processing, the script outputs a styled HTML response. Depending on a $success boolean, the message text color changes dynamically (green #4CAF50 for success, red #f44336 for errors), clearly informing the user of the outcome.

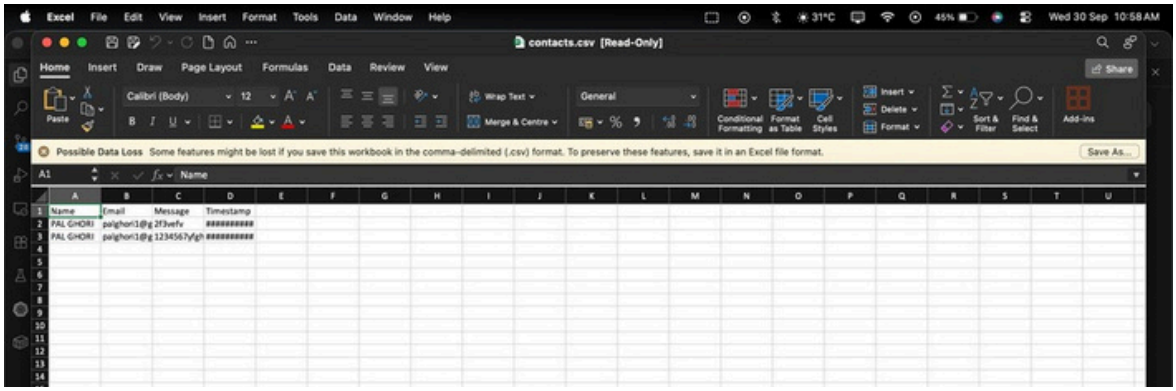
B. Implementation

Transitioned the project from purely frontend to full-stack. Configured forms in register.html and contact.html to send POST requests. Created process_register.php (which sanitizes data and saves to a .json file) and process_contact.php (which sanitizes data and appends it to a .csv file).

C. Evidence of Work (Screenshots)




Safari File Edit View History Bookmarks Window Help
Wed 30 Sep 10:56 AM
localhost
StudentHub
Home Contact
Contact Status
Message sent successfully! Data saved to CSV.
Back to Contact


Excel File Edit View Insert Format Tools Data Window Help
Wed 30 Sep 10:58 AM
contacts.csv [Read-Only]
Possible Data Loss Some features might be lost if you save this workbook in the comma-delimited (.csv) format. To preserve these features, save it in an Excel file format.
Save As...
Name
Email
Message
Timestamp
PAL GHORI
polghori1@g
2f3vefv
1234567yjgh

D. GitHub Repository Link:

https://github.com/palghori/STUDENT-HUB-PORTAL-WDF/tree/main/practical%207

E. Challenges Faced:

- Understanding the bridge between the frontend (HTML forms) and the backend (PHP $_POST array).
- Dealing with server environments (XAMPP/Apache) and figuring out file permission issues (chmod 777 on macOS) so that PHP had the correct read/write access to create the DATA directory and the JSON/CSV files.
- Ensuring security by heavily sanitizing inputs (htmlspecialchars, strip_tags, FILTER_SANITIZE_EMAIL) to prevent XSS (Cross-Site Scripting) attacks.

F. Learning Outcomes:

Mastered the fundamentals of Backend Web Development: handling HTTP POST requests, server-side data sanitization, file I/O operations in PHP (working with fopen, fputcsv, and json_encode), and running a local server environment (XAMPP).

Faculty Remarks:

Practical - 8

MySQL Schema Design, ER Model, PDO Connectivity, and Prepared Statements

Problem Definition Design the database for StudentHub with students, events and registrations. Create PHP database connection using MYSQL statements.

A. Theory

Is the schema normalized properly?

Yes. The schema is properly structured into separate tables such as students, events, and registrations, which reduces duplicate data. The registrations table handles the many-to-many relationship between students and events.

Are primary keys and foreign keys correctly defined?

Yes. Each table has its own Primary Key, such as student_id, event_id, and registration_id. The registrations table uses student_id and event_id as Foreign Keys to connect the related tables.

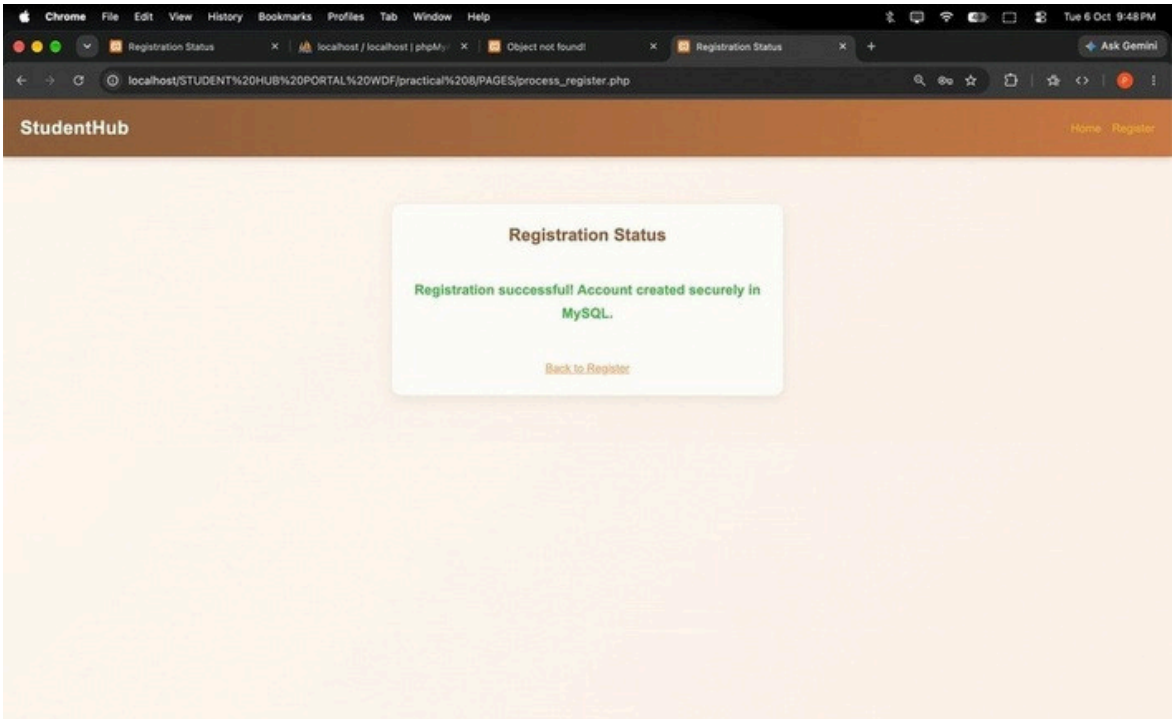
Is the database connection handled securely?

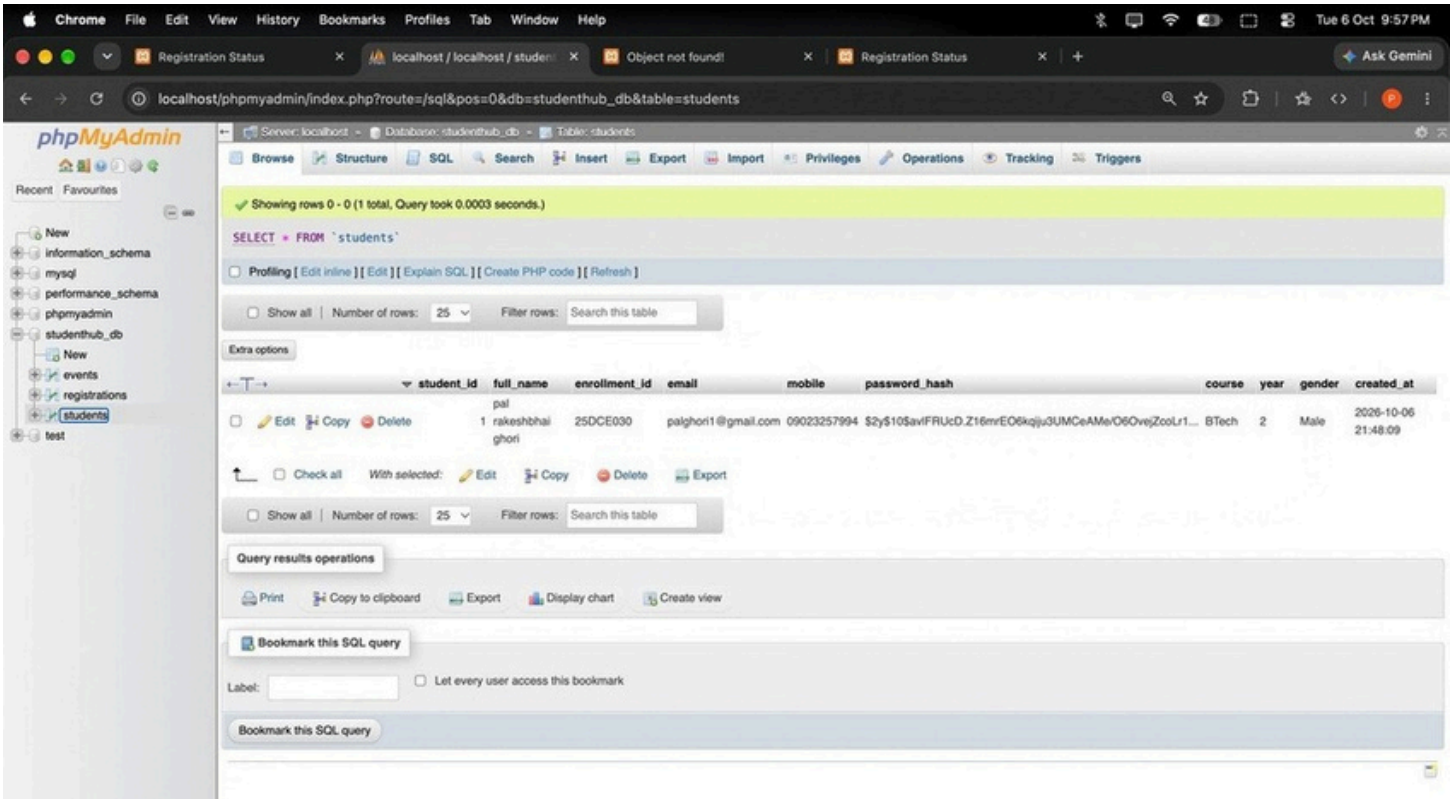
Yes. Practical 8 uses PDO, prepared statements, and prepare()/execute() instead of directly inserting user input into SQL queries. This helps prevent SQL Injection. Passwords are also stored using password_hash() rather than as plain text.

B. Implementation

- Created three MySQL tables: students, events, and registrations.
- Defined Primary Keys and Foreign Keys to establish relationships between the tables.
- Connected PHP with MySQL using PDO.
- Used prepared statements with prepare() and execute() to insert and retrieve data securely.
- Used password_hash() to store passwords securely instead of saving plain-text passwords.
- Stored student registration and event-related data in the relational database.

C. Evidence of Work (Screenshots)




Chrome File Edit View History Bookmarks Profiles Tab Window Help
Tue 6 Oct 9:57 PM
Registration Status
localhost / localhost / studen
Object not found!
Registration Status
Ask Gemini
localhost/phpmyadmin/index.php?route=/sql&pos=0&db=studenthub_db&table=students
phpMyAdmin
Recent Favourites
New
information_schema
mysql
performance_schema
phpmyadmin
studenthub_db
New
events
registrations
students
test
Server: localhost » Database: studenthub_db » Table: students
Browse Structure SQL Search Insert Export Import Privileges Operations Tracking Triggers
Showing rows 0 - 0 (1 total, Query took 0.0003 seconds.)
SELECT * FROM `students`
Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]
Show all | Number of rows: 25 Filter rows: Search this table
Extra options
student_id full_name enrollment_id email mobile password_hash course year gender created_at
Edit Copy Delete 1 pal rakeshbhai ghori 25DCE030 palghori1@gmail.com 09023257994 $2y$10$avIFRUcD.Z16mrEO6kqjju3UMCeAMe/O6OvejZcoLr1... BTech 2 Male 2026-10-06 21:48:09
Check all With selected: Edit Copy Delete Export
Show all | Number of rows: 25 Filter rows: Search this table
Query results operations
Print Copy to clipboard Export Display chart Create view
Bookmark this SQL query
Label:
Let every user access this bookmark
Bookmark this SQL query

D. GitHub Repository Link:

https://github.com/palghori/STUDENT-HUB-PORTAL-WDF/tree/main/practical%208

E. Challenges Faced:

- Understanding the relationship between students, events, and registrations.
- Correctly defining Primary Keys and Foreign Keys.
- Handling database connection errors using PDO.
- Writing SQL queries with prepared statements and placeholders.
- Avoiding duplicate records and maintaining data consistency.
- Understanding the difference between password hashing and normal encryption.

F. Learning Outcomes:

- Learned how to design a basic relational database schema.
- Understood Primary Keys, Foreign Keys, and table relationships.
- Learned how PHP communicates with MySQL using PDO.
- Learned how prepared statements prevent SQL Injection.
- Learned secure password storage using password_hash().
- Gained practical knowledge of storing and managing dynamic website data in a real database instead of JSON/CSV files.

Faculty Remarks:

Practical - 9

Secure User Registration with Database Insert, Duplicate Email Check, and Password Hashing

Problem Definition Implement secure user registration by inserting user details into MySQL using MYSQLi. Validate input on frontend and backend, check duplicate email/username, hash passwords using password_hash(), and display appropriate messages.

A. Theory

1. Is password_hash() used before storing passwords?

Yes. In process_register.php, before the INSERT query runs, the raw password is passed through password_hash($raw_password, PASSWORD_DEFAULT). The hashed version is then stored in the password_hash column — the plain text password is never saved to the database.

2. Is duplicate email/username checked?

Yes. Before inserting, a SELECT query runs on the students table:

SELECT * FROM students WHERE email = '$email' OR enrollment_id = '$enrollment'

If mysqli_num_rows() returns more than 0, the script stops and shows: "An account with this Email or Enrollment ID already exists!"

3. Are frontend and backend validations consistent?

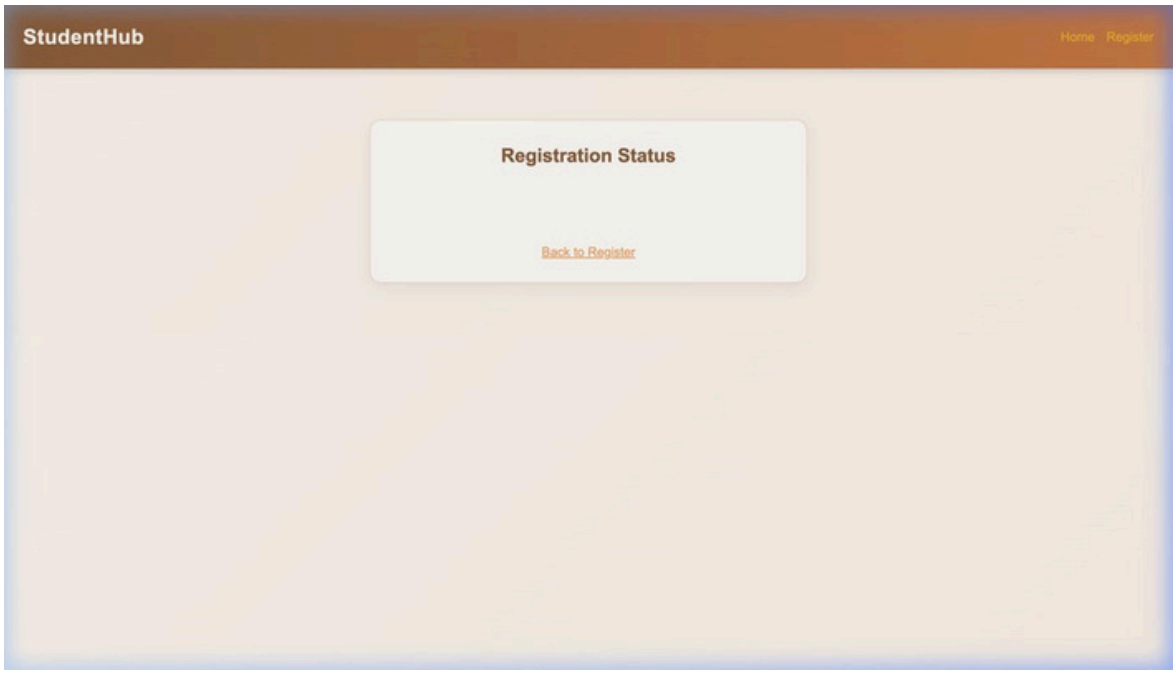
Yes. The HTML register.html uses required attributes, type="email" for the email field, and input constraints as frontend validation. The PHP backend then performs the same checks again — empty field check, email format validation, and duplicate check — ensuring both layers are aligned.

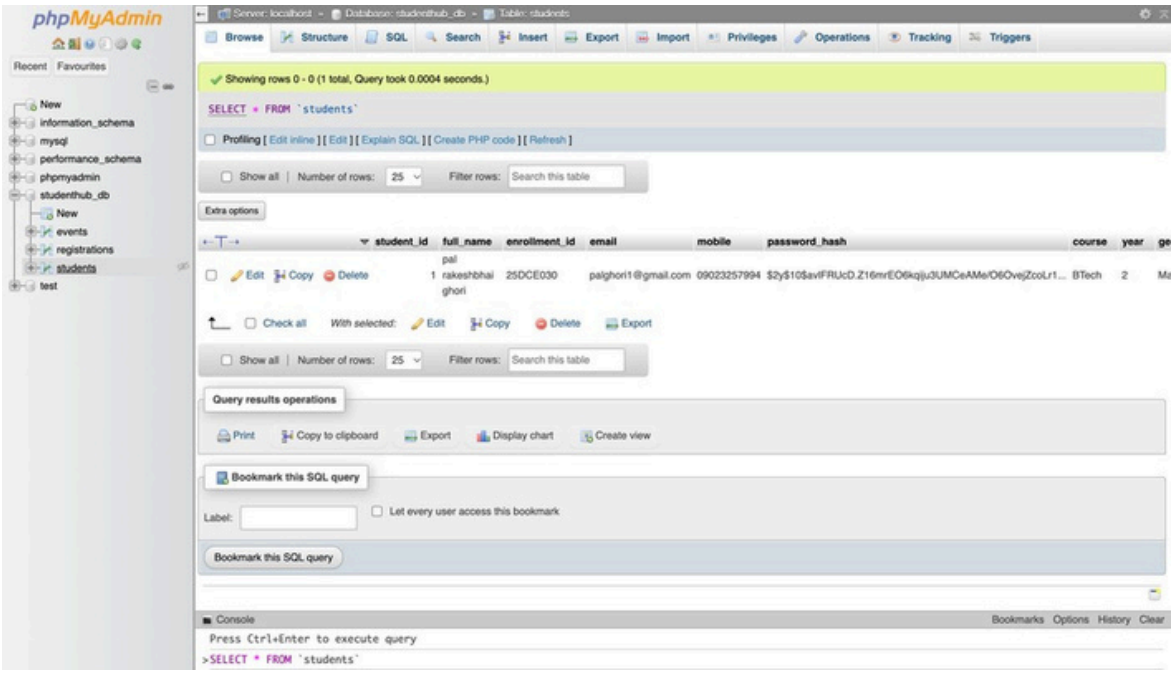
B. Implementation

The implementation focused on secure user registration using MySQLi (basic procedural style):

File	Role
db_connect.php	Connects to MySQL using mysqli_connect()
process_register.php	Sanitizes → Validates → Checks duplicate → Hashes password → Inserts into DB
register.html	Frontend form with basic HTML5 validation

C. Evidence of Work (Screenshots)




phpMyAdmin
Recent Favourites
New
information_schema
mysql
performance_schema
phpmyadmin
studenthub_db
New
events
registrations
students
test
Server: localhost » Database: studenthub_db » Table: students
Browse Structure SQL Search Insert Export Import Privileges Operations Tracking Triggers
Showing rows 0 - 0 (1 total, Query took 0.0004 seconds.)
SELECT * FROM `students`
Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]
Show all | Number of rows: 25 Filter rows: Search this table
Extra options
student_id full_name enrollment_id email mobile password_hash course year
Edit Copy Delete 1 pal rakeshbhai ghori 25DCE030 palghori1@gmail.com 09023257994 $2y$10$avIFRUcD.Z16mrEO6kqjju3UMCeAMe/O6OvejZcoLr1... BTech 2
Check all With selected: Edit Copy Delete Export
Show all | Number of rows: 25 Filter rows: Search this table
Query results operations
Print Copy to clipboard Export Display chart Create view
Bookmark this SQL query
Label:
Let every user access this bookmark
Bookmark this SQL query
Console
Bookmarks Options History Clear
Press Ctrl+Enter to execute query
>SELECT * FROM `students`

D. GitHub Repository Link:

https://github.com/palghori/STUDENT-HUB-PORTAL-WDF/tree/main/practical%209

E. Challenges Faced:

MySQLi vs PDO: Understanding the difference between procedural mysqli_query() (Practical 9) and object-oriented PDO::prepare() (Practical 8). MySQLi is simpler but less flexible.
Duplicate Check Logic: Writing a single SELECT that checks both email AND enrollment_id in one query using OR.
Password Handling: Understanding why the raw password must be passed directly to password_hash() without mysqli_real_escape_string() first — escaping can corrupt the hash.

F. Learning Outcomes:

MySQLi Procedural Connection — Learned how to use mysqli_connect(), mysqli_query(), and mysqli_num_rows() to interact with a MySQL database.
Server-Side Sanitization — Used mysqli_real_escape_string() to protect against basic SQL injection before building queries.
Secure Password Storage — Understood why plain text passwords must never be stored, and how password_hash() with PASSWORD_DEFAULT creates a secure one-way hash.
Duplicate Prevention — Implemented a pre-insert check using SELECT to prevent duplicate accounts before they cause database errors.
Form Validation (Two Layers) — Learned that frontend validation (HTML5) is for user experience, but backend validation is mandatory for security since frontend can be bypassed.

Faculty Remarks:

Practical - 10

Secure Login/Logout with Sessions, Role-Based Access, Timeout, and Remember-Me Token

Problem Definition Develop secure login and logout functionality using PHP sessions. Verify password using password_verify(), regenerate session ID after login, restrict dashboard access, implement role-based redirection, session timeout, and optional secure remember-me token.

A. Theory

1. Are sessions started and destroyed securely?

Yes. session_start() is called at the very top of process_login.php. On logout, logout.php does three things in order: clears $_SESSION = array(), calls session_destroy(), and deletes the remember-me cookie — this is the correct and secure way to fully terminate a session.

2. Is password_verify() used correctly?

Yes. After fetching the user's row from the database, the script calls password_verify($password, $user['password_hash']). The raw typed password is passed as the first argument and the stored hash from MySQL as the second — exactly the correct usage.

3. Is unauthorized dashboard access blocked?

Yes. check_session.php checks for $_SESSION['student_id']. If it's not set, the user is redirected to login.html?msg=pleaselogin immediately with exit(). Any page that includes this file becomes protected.

4. Is session timeout implemented?

Yes. At the top of process_login.php, if $_SESSION['last_active'] exists and is more than 900 seconds (15 minutes) old, the session is destroyed and the user is sent to login.html?msg=timeout. The last_active value is refreshed on every successful request.

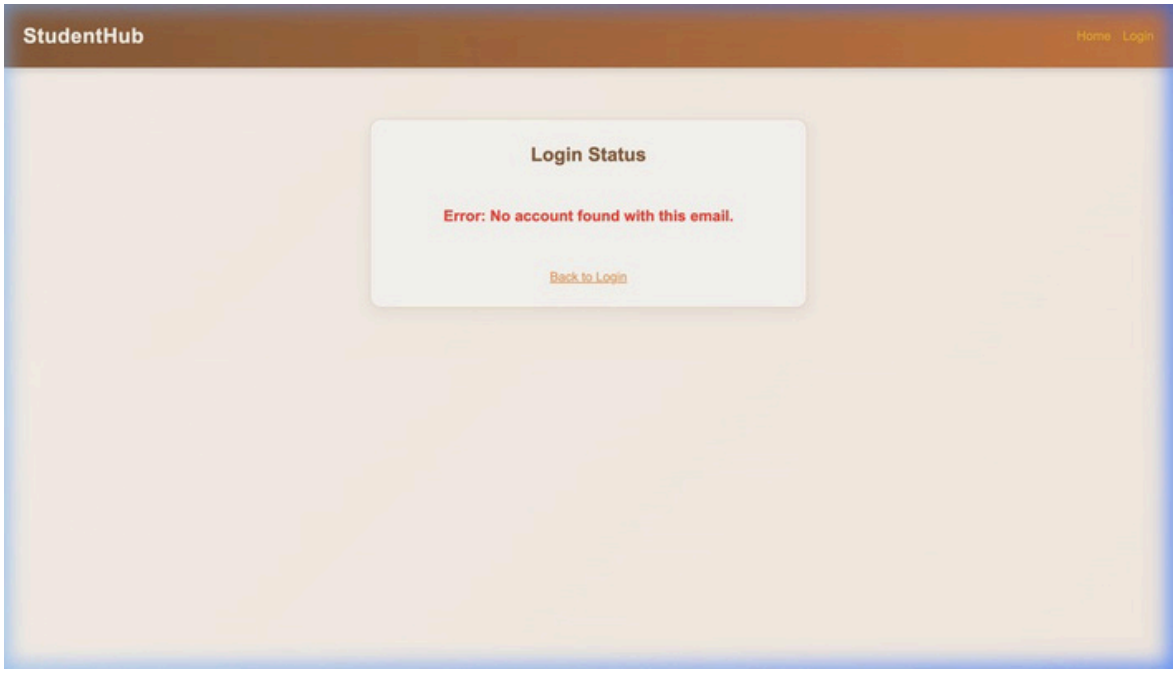
5. Are admin and student users redirected to appropriate dashboards?

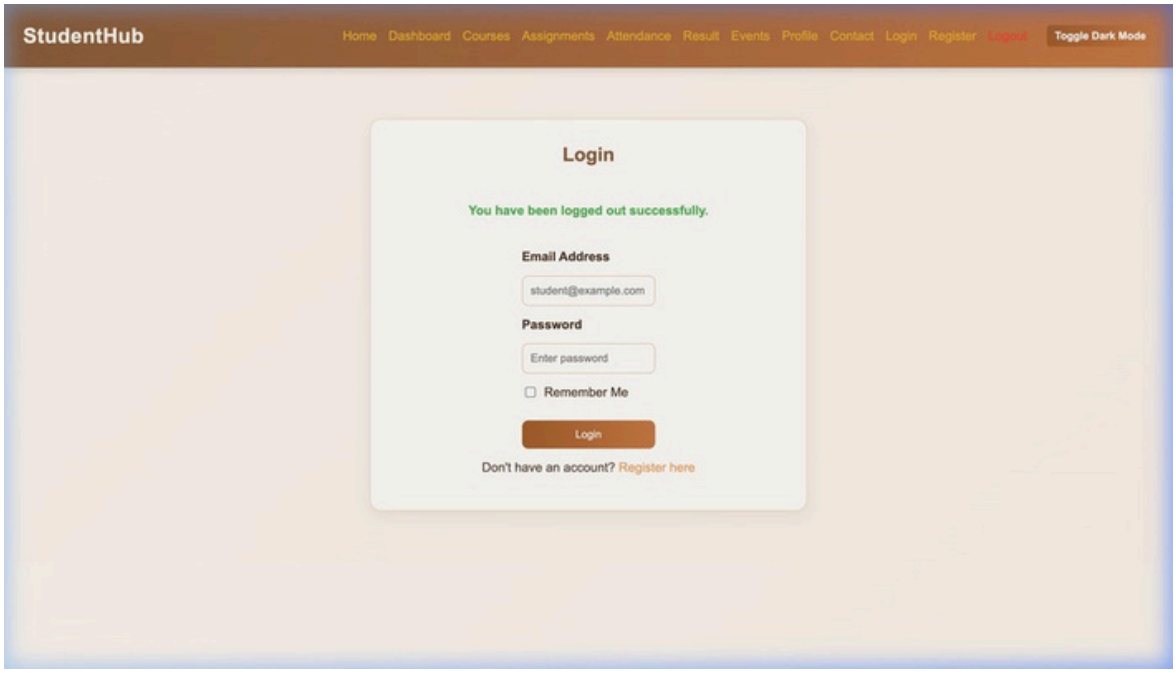
Yes. After a successful login, the script checks $user['role']:
If admin → redirects to dashboard.html?welcome=admin
If student → redirects to dashboard.html?welcome=student

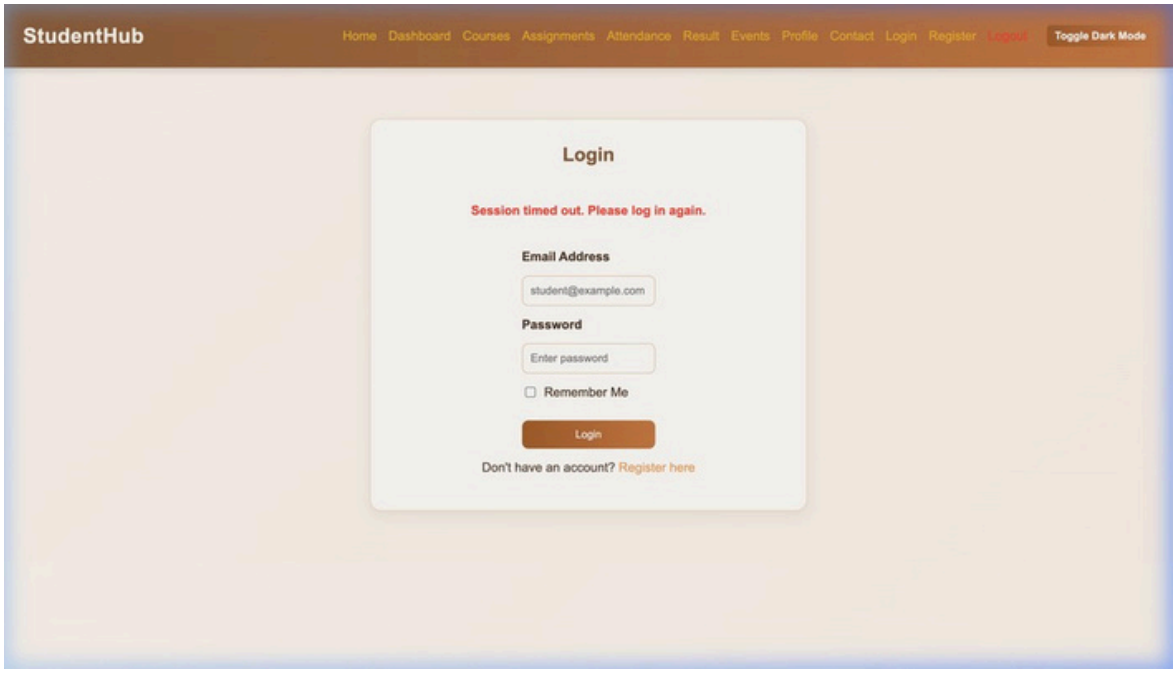
B. Implementation

File	What It Does
login.html	Frontend login form with Email, Password, Remember Me checkbox
process_login.php	Verifies credentials with password_verify(), sets session, handles role-based redirect
logout.php	Clears session, deletes cookie, redirects with success message
check_session.php	Reusable session guard — protects any page that includes it
DATA/update_schema.sql	Adds role column to students table for admin/student distinction

C. Evidence of Work (Screenshots)




StudentHub
Home Dashboard Courses Assignments Attendance Result Events Profile Contact Login Register Logout
Toggle Dark Mode
Login
You have been logged out successfully.
Email Address
student@example.com
Password
Enter password
Remember Me
Login
Don't have an account? Register here


StudentHub
Home Dashboard Courses Assignments Attendance Result Events Profile Contact Login Register Logout
Toggle Dark Mode
Login
Session timed out. Please log in again.
Email Address
student@example.com
Password
Enter password
Remember Me
Login
Don't have an account? Register here

D. GitHub Repository Link:

https://github.com/palghori/STUDENT-HUB-PORTAL-WDF/tree/main/practical%210

E. Challenges Faced:

1. Session ID Regeneration — Understanding why session_regenerate_id(true) must be called after verifying login but before storing session variables, to prevent session fixation attacks.
2. Remember Me Cookie — Setting the cookie expiry correctly (time() + 7 * 24 * 60 * 60) and deleting it on logout by setting expiry to a past time (time() - 3600).
3. Password Not Escaped — mysqli_real_escape_string() must NOT be applied to the password before passing it to password_verify(). Escaping the password first would corrupt the comparison.

F. Learning Outcomes:

Learned how PHP sessions work — starting, storing variables, and destroying them.
Understood password_verify() and why it pairs with password_hash() for secure authentication.
Learned session timeout logic using time() and last_active tracking.
Understood role-based redirection — different users get sent to different pages based on their role in the database.
Learned the Remember Me pattern using browser cookies (setcookie()).

Faculty Remarks:

Practical - 11

Student Management CRUD with Search and Filter

Problem Definition: Create a student management module to add, view, edit, delete, search and filter student records. All database operations must use MYSQLi statements and clear success/failure messages.

A. Theory

1. Are SELECT, INSERT, UPDATE, and DELETE operations working? Yes — all four CRUD operations are implemented:

SELECT → students.php fetches and displays all students from the students table.
INSERT → add_student.php adds a new student with duplicate check and password hashing.
UPDATE → edit_student.php pre-fills the form using SELECT, then updates the record on submit.
DELETE → students.php handles ?delete=id in the URL, runs DELETE FROM students WHERE student_id = $id.

2. Is search by name/email/course implemented?

Yes — the search form on students.php uses a LIKE '%search%' query on full_name and email, plus a dropdown filter for course. Both can be used together at the same time.

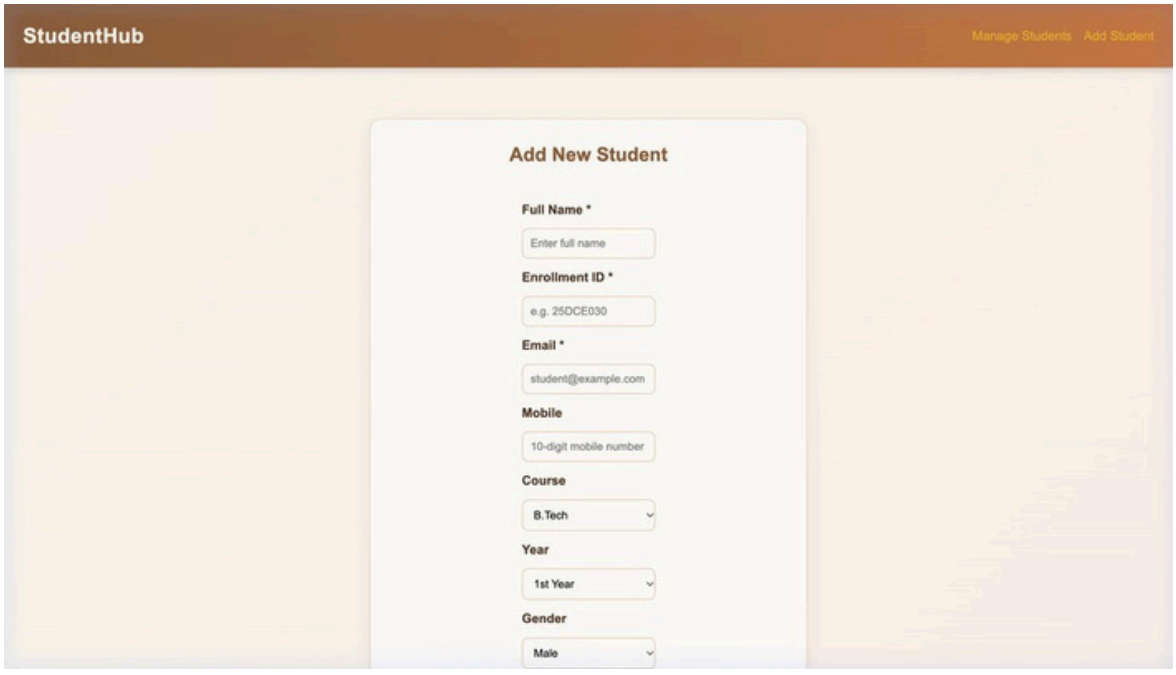
3. Are delete operations confirmed before execution?

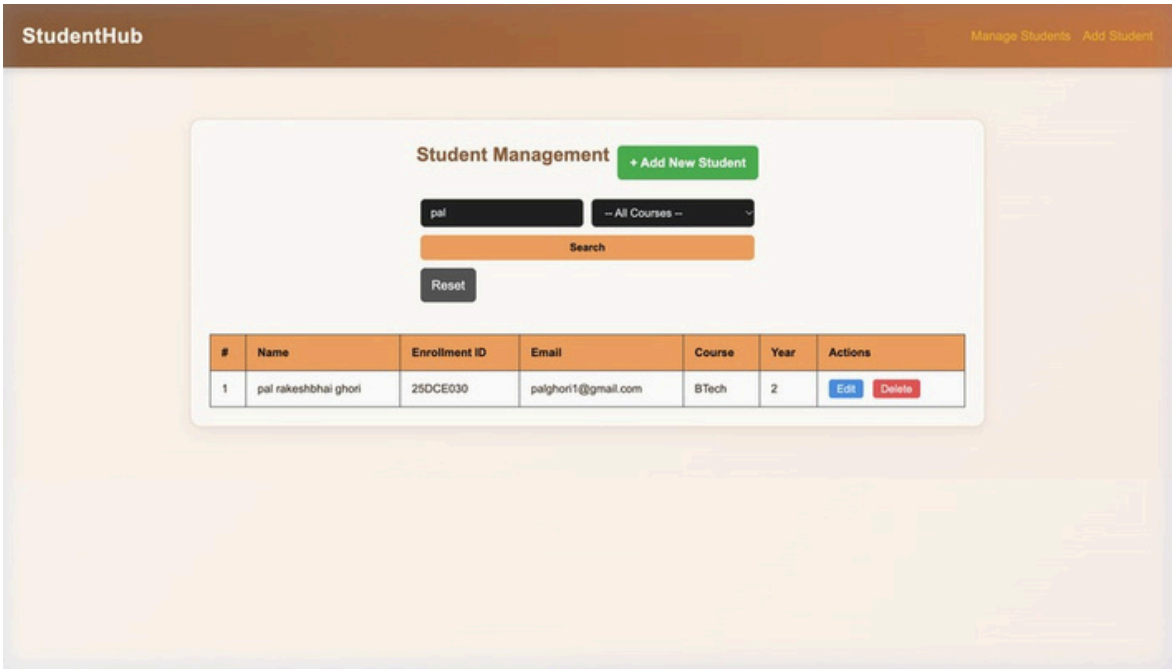
Yes — the Delete button uses JavaScript's built-in onclick="return confirm('Are you sure you want to delete this student?')". If the user clicks Cancel, the deletion does not happen.

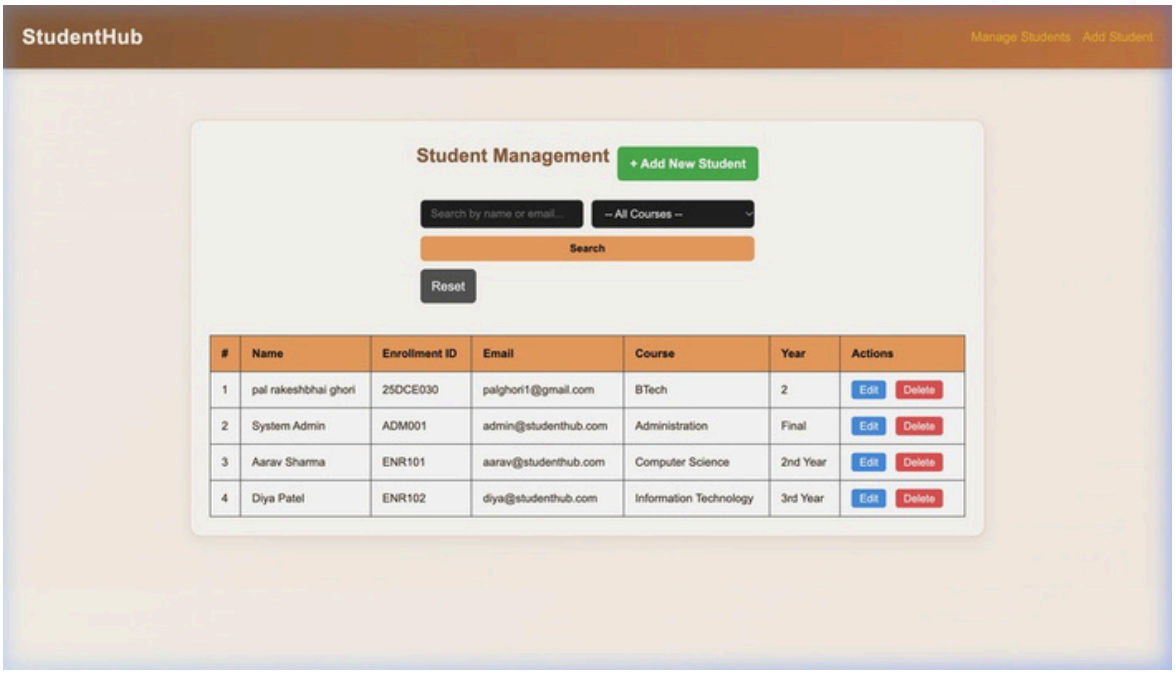
B. Implementation

File	What It Does
db_connect.php	Reused from Practical 9 — basic MySQLi
students.php	Lists all students, handles
add_student.php	Form to INSERT a new student
edit_student.php	Pre-fills form via SELECT, updates via UPDATE

C. Evidence of Work (Screenshots)




StudentHub
Manage Students
Add Student
Student Management
+ Add New Student
pal
-- All Courses --
Search
Reset
Name Enrollment ID Email Course Year Actions
1 pal rakeshbhai ghori 25DCE030 palghori1@gmail.com BTech 2 Edit Delete


StudentHub
Manage Students
Add Student
Student Management
+ Add New Student
Search by name or email...
-- All Courses --
Search
Reset
Name Enrollment ID Email Course Year Actions
1 pal rakeshbhai ghori 25DCE030 palghori1@gmail.com BTech 2 Edit Delete
2 System Admin ADM001 admin@studenthub.com Administration Final Edit Delete
3 Aarav Sharma ENR101 aarav@studenthub.com Computer Science 2nd Year Edit Delete
4 Diya Patel ENR102 diya@studenthub.com Information Technology 3rd Year Edit Delete

D. GitHub Repository Link:

https://github.com/palghori/STUDENT-HUB-PORTAL-WDF/tree/main/practical%211

E. Challenges Faced:

1.Building dynamic SQL queries — Adding WHERE 1=1 to the base query made it easy to optionally append search and filter conditions without breaking the syntax.
2.Pre-filling the edit form — Fetching the current row via SELECT and displaying its values as value="..." in the HTML form required careful use of htmlspecialchars() to avoid breaking the HTML if a student's name has quotes.
3.Preventing accidental deletes — A direct link without confirmation would delete data immediately, so onclick="return confirm(...)" was added as a safety net.

F. Learning Outcomes:

1.Learned all four CRUD operations (Create, Read, Update, Delete) using MySQLi procedural style.
2.Understood how to build dynamic SQL queries with optional WHERE clauses for search and filter.
3.Learned how LIKE '%keyword%' works for pattern-based searching in MySQL.
4.Understood the importance of confirming destructive actions before executing them.
5.Learned how to pre-fill HTML forms from database data to create a smooth edit experience.

Faculty Remarks:

Practical - 12

Event Management CRUD with Poster Upload and File Validation

Problem Definition : Develop event management functionality for admin users. Admin can add, view, update, delete, and upload event posters with file type and file size validation.

A. Theory

1.Are add, update, and delete functionalities implemented correctly?

Yes — all 3 are implemented:

- Add → add_event.php runs INSERT INTO events
- Update → edit_event.php pre-fills from SELECT, then runs UPDATE events
- Delete → events.php handles ?delete=id, runs DELETE FROM events, and also deletes the poster file from the UPLOADS folder using unlink()

2. Is the UI linked with the database correctly?

Yes — events.php fetches all events using SELECT * FROM events and displays them in an HTML table with poster thumbnails, title, category, date, and action buttons (Edit/Delete) — all pulled directly from the MySQL database.

3.Are poster uploads validated for type and size?

Yes — before moving the uploaded file, the script checks:

- File type: only jpg, jpeg, png, gif are accepted using pathinfo() and in_array()
- File size: must be under 2MB (2 × 1024 × 1024 bytes)
- If either check fails, an error message is displayed and no file is saved

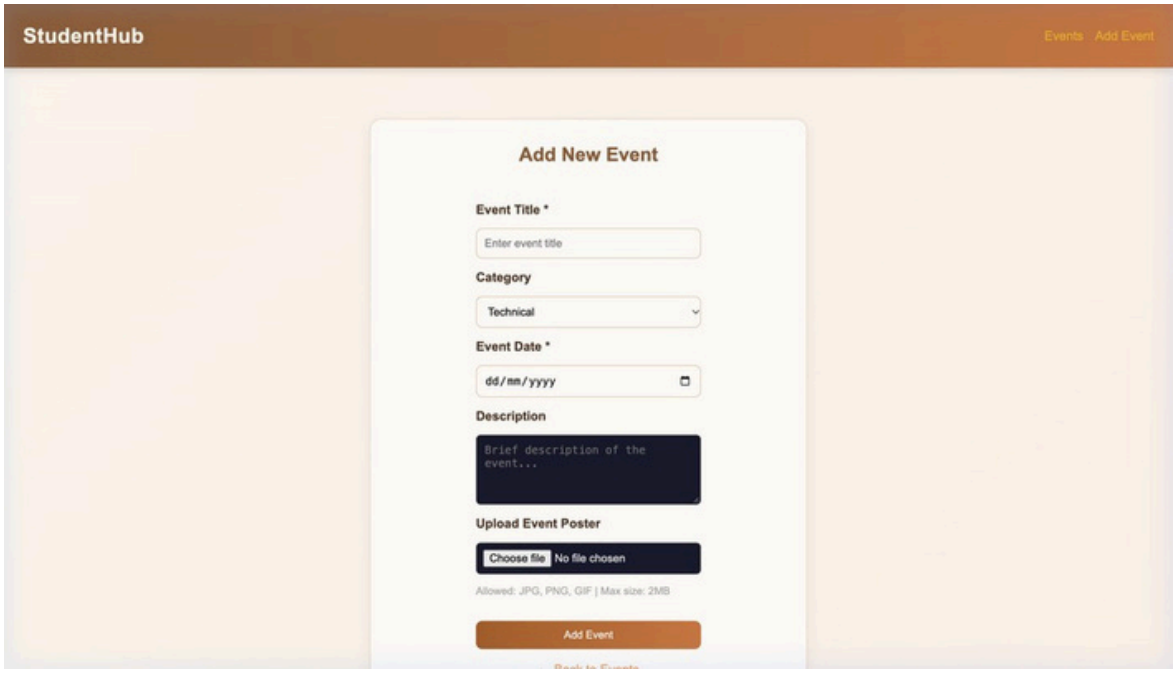
4. Are success and failure messages displayed?

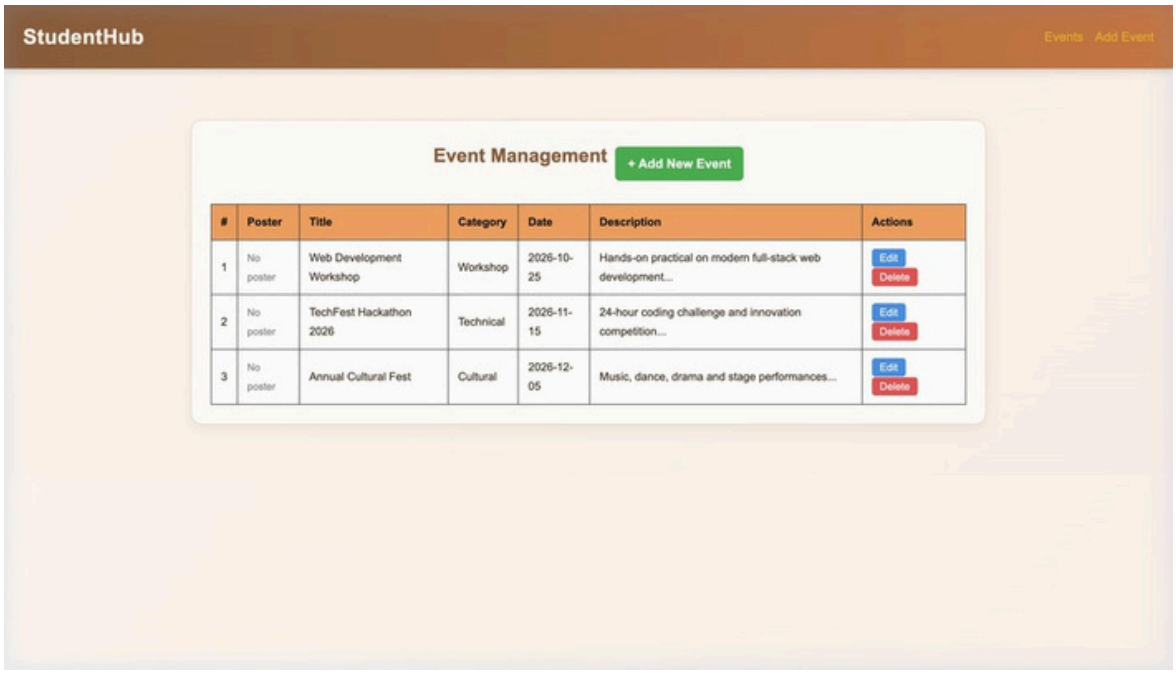
Yes — all three pages display color-coded messages: green for success, red for errors. Error cases include wrong file type, file too large, empty required fields, and database errors.

B. Implementation

db_connect.php	Reused — basic MySQLi
events.php	Lists all events, handles DELETE
add_event.php	Form to INSERT event + upload
edit_event.php	Pre-filled form for UPDATE event +
update_events_table.sql	Adds poster column to existing
UPLOADS/	Folder where uploaded poster

C. Evidence of Work (Screenshots)




StudentHub
Events
Add Event
Event Management
+ Add New Event
Poster Title Category Date Description Actions
1 No poster Web Development Workshop Workshop 2026-10-25 Hands-on practical on modern full-stack web development... Edit Delete
2 No poster TechFest Hackathon 2026 Technical 2026-11-15 24-hour coding challenge and innovation competition... Edit Delete
3 No poster Annual Cultural Fest Cultural 2026-12-05 Music, dance, drama and stage performances... Edit Delete

D. GitHub Repository Link:

https://github.com/palghori/STUDENT-HUB-PORTAL-WDF/tree/main/practical%212

E. Challenges Faced:

1.File Upload with enctype — The HTML form must use enctype="multipart/form-data" for file uploads to work. Without this, $_FILES is always empty — a very common beginner mistake.
2.Unique Filenames — Using time() prefix (1728835200_poster.jpg) prevents two uploads from overwriting each other if they have the same filename.
3.Deleting Old Poster on Update — When a new poster is uploaded during edit, the old file must be deleted from disk using unlink(). Forgetting this would leave orphan files in the UPLOADS folder.
4.UPLOADS folder permissions — The folder must have chmod 777 so that PHP (running as the Apache user) is allowed to write files into it.

F. Learning Outcomes:

1.Learned complete CRUD for a different entity (Events instead of Students) — reinforcing the pattern.
2.Learned file uploads in PHP using $_FILES, move_uploaded_file(), and enctype="multipart/form-data".
3.Understood file validation — checking both file extension (pathinfo) and file size ($_FILES['poster']['size']).
4.Learned how to manage uploaded files — creating unique names with time(), storing only the filename in the database, and deleting files with unlink().
5.Learned to display thumbnail previews of uploaded images in the HTML table.

Faculty Remarks:

Practical - 13

PHP JSON API and AJAX/Fetch-Based CRUD without Page Reload

Problem Definition : Create simple PHP JSON API endpoints for events or students and consume them using Fetch API. Perform list, add, update, and delete operations asynchronously without full page reload.

A. Theory

1. Do PHP endpoints return valid JSON responses?

Yes. api.php starts with header("Content-Type: application/json") and every response uses json_encode() with a consistent format:

```
{ "success": true/false, "message": "...", "data": [...] }
```

2.Are GET and POST requests handled properly?

Yes — all 4 HTTP methods are handled inside api.php using $_SERVER['REQUEST_METHOD']:

- GET → Returns all events as JSON array
- POST → Reads JSON body with file_get_contents("php://input") and inserts into DB
- PUT → Reads JSON body and updates the matching event
- DELETE → Reads JSON body and deletes the matching event

3.Are loading, success, and error states shown on the frontend?

Yes — three UI states are handled in app.js:

- Loading: The #loading div shows ⏳ Loading... while any Fetch request is in progress
- Success: Green text message appears for 3 seconds (auto-hides)
- Error: Red text message appears for 3 seconds (auto-hides)

4.Is input validation performed before database operations?

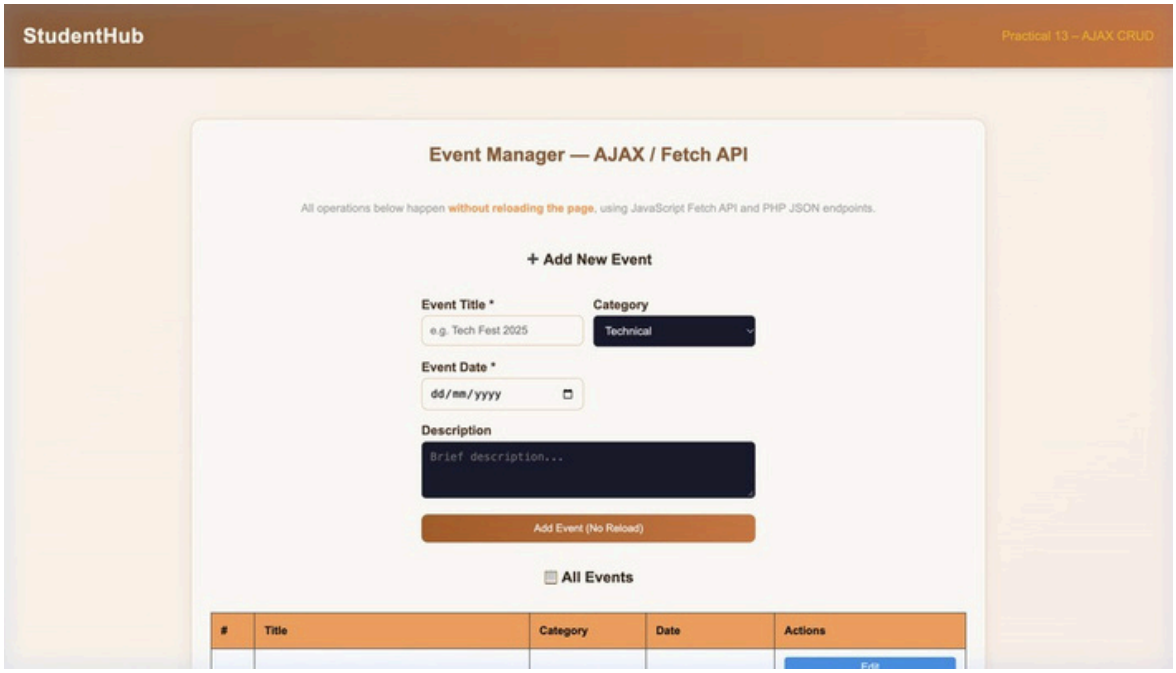
Yes — validation happens on both sides:

- Frontend (JS): Checks if Title and Date are empty before even sending the Fetch request
- Backend (PHP): Also checks for empty fields after receiving the JSON, returns an error JSON response if validation fails — so the API is safe even if someone bypasses the frontend

B. Implementation

api.php	PHP JSON API — handles GET, POST, PUT, DELETE requests
index.html	Frontend page — Add form, Events table, Edit form
app.js	JavaScript — all Fetch API calls, table rendering, messages
db_connect.php	Reused — basic MySQLi connection
CSS/script.css	Reused styles from previous practicals

C. Evidence of Work (Screenshots)



D. GitHub Repository Link:

https://github.com/palghori/STUDENT-HUB-PORTAL-WDF/tree/main/practical%213

E. Challenges Faced:

1.Reading JSON request body in PHP — Unlike regular form POSTs where data comes in $_POST, a Fetch API request sends JSON. You must read it using file_get_contents("php://input") and then json_decode(). This is a key difference for beginners to understand.
2.PUT and DELETE in PHP — PHP doesn't natively support $_PUT or $_DELETE superglobals, so both must use file_get_contents("php://input") — same as POST.
3.Preventing Page Reload — The form's onsubmit handler must call event.preventDefault() first. Without this, the browser reloads the page on every form submit, defeating the purpose of AJAX.
4.Updating the table without reload — After every successful add/edit/delete, loadEvents() is called again which re-fetches all events from the API and re-renders the table — giving a seamless real-time feel.

F. Learning Outcomes:

1.Learned what a JSON API is — a PHP file that returns structured JSON data instead of HTML.
2.Understood the Fetch API in JavaScript — how to make GET, POST, PUT, DELETE requests from the browser.
3.Learned how to read JSON request bodies in PHP using file_get_contents("php://input").
4.Understood async/await — using async function and await to handle asynchronous fetch calls without complex callback chains.
5.Learned how to do two-layer validation — frontend (JavaScript) and backend (PHP) for the same data.

Faculty Remarks:

Practical - 14

Admin Dashboard with Role-Based Access Control, Dynamic Menu, and Audit Log

Problem Definition : Develop an admin dashboard that displays user statistics, student count, event count and latest registrations. Restrict access to admin users only and show dynamic menu options based on role.

A. Theory

1. Are dashboard values loaded dynamically from the database?

Yes. All metrics on the admin dashboard (Active Students, Total Events, Total Registrations, Admin Accounts, and Audit Log count) are calculated dynamically using SQL queries (SELECT COUNT(*) FROM ...). The "Latest Registrations" table is also fetched dynamically using an INNER JOIN across registrations, students, and events.

2. Is access restricted to admin users?

Yes. Every administrative page checks $_SESSION['role'] === 'admin' at the very top of the script. If an unauthorized user (student or guest) tries to access the page directly via URL, they are immediately blocked with header("Location: access_denied.php"); exit();.

3. Are role-based menus working?

Yes. The modular navbar.php checks the session role:

- When logged in as Admin: Displays Dashboard, Students, Events, Audit Logs, and Logout (Admin).
- When logged in as Student: Displays Student Home, View Events, Try Admin Area, and Logout (Student).
- When Guest: Displays Login only.

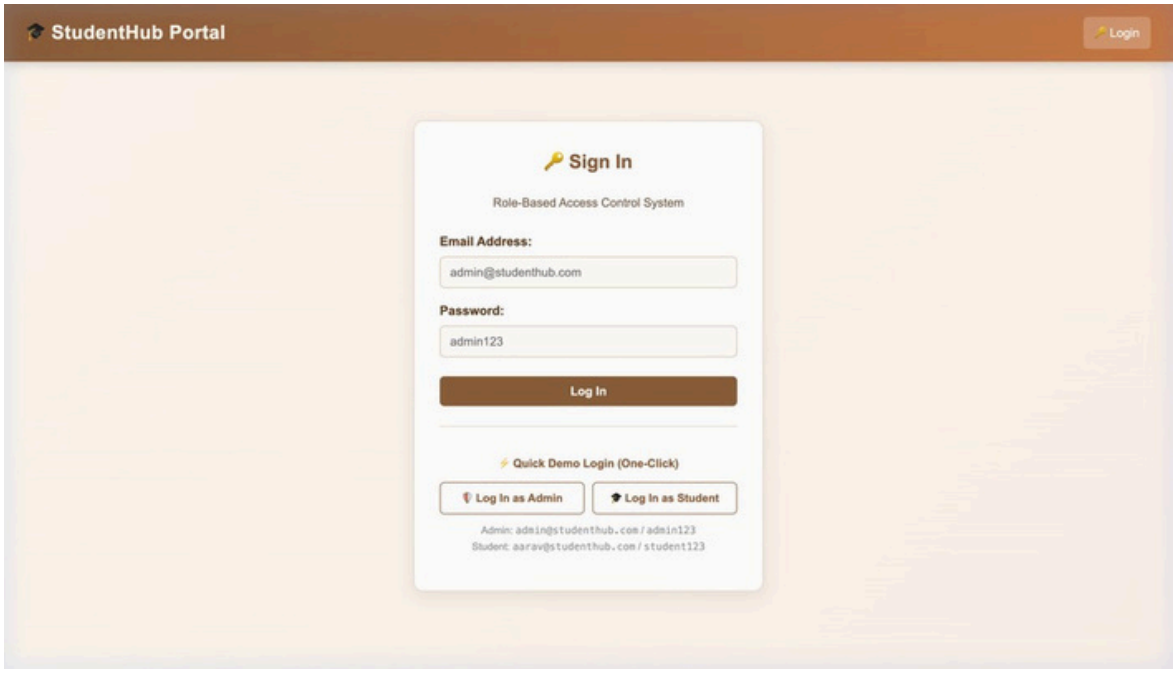
4. Are important admin actions recorded in audit_logs?

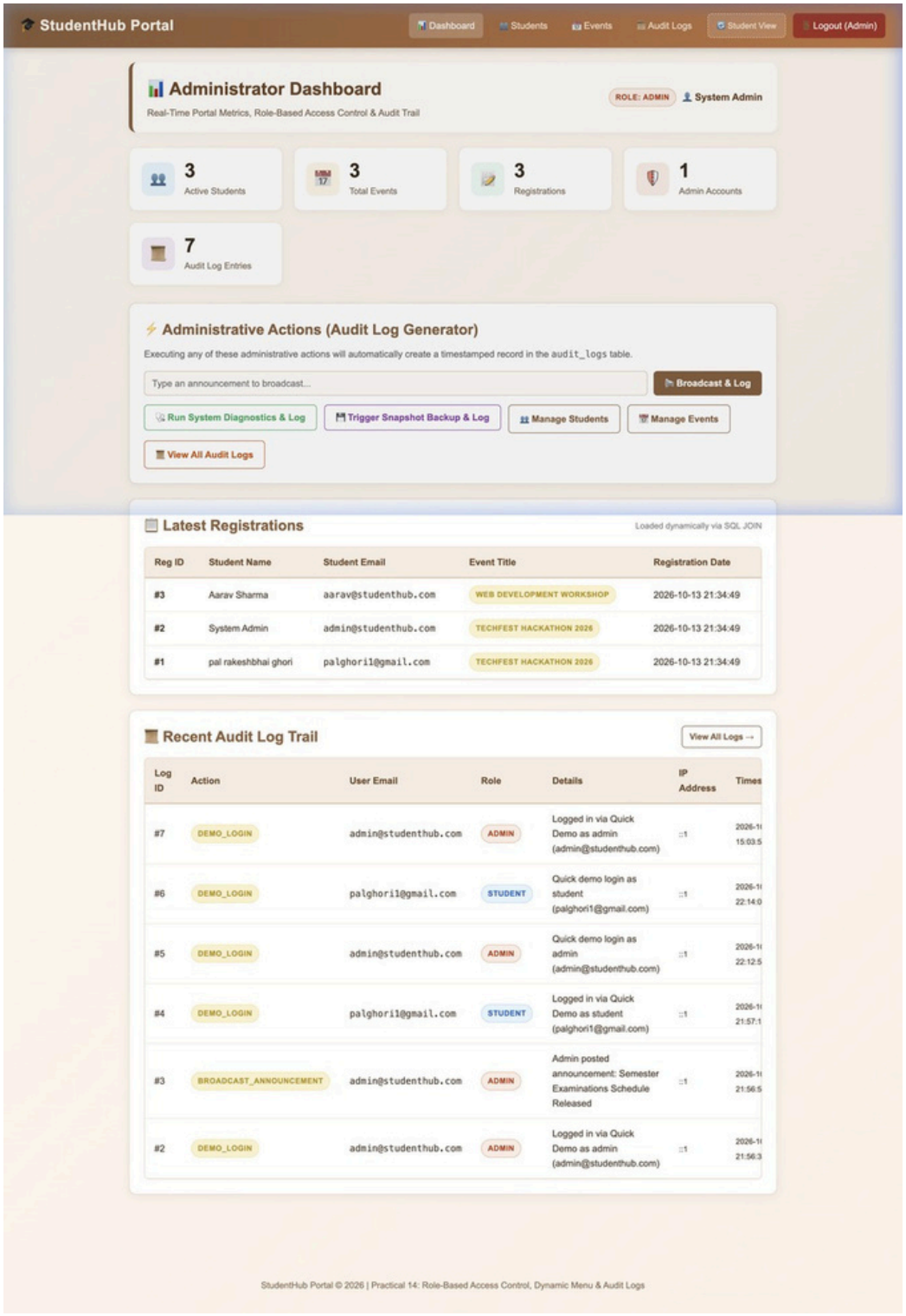
Yes. A dedicated audit_logs table records actions using record_audit_log($conn, $action, $details). Actions such as Admin Login, Logout, Announcement broadcasts, diagnostics, event addition/deletion, and role changes are stored with the user email, role, action type, description, client IP, and timestamp.

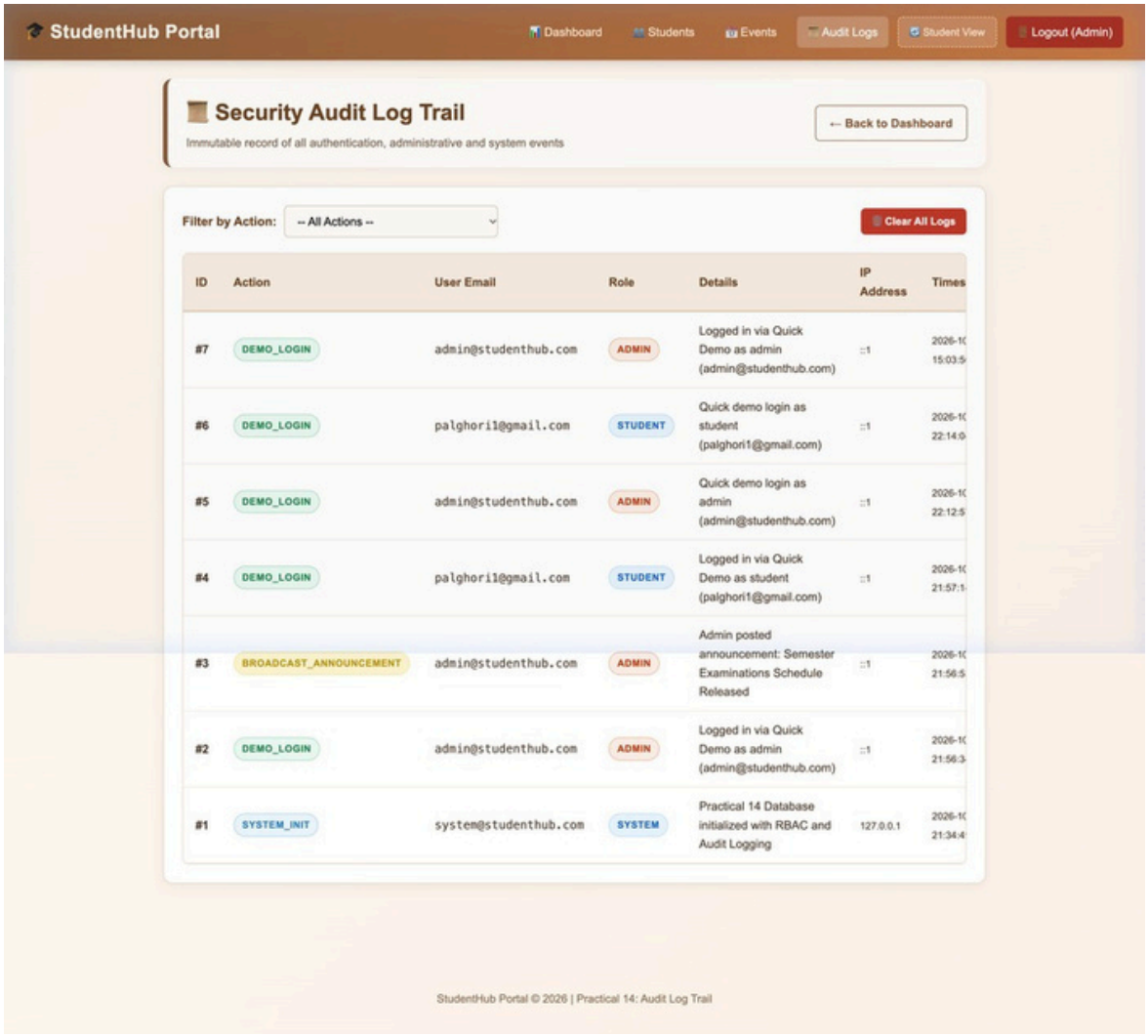
B. Implementation

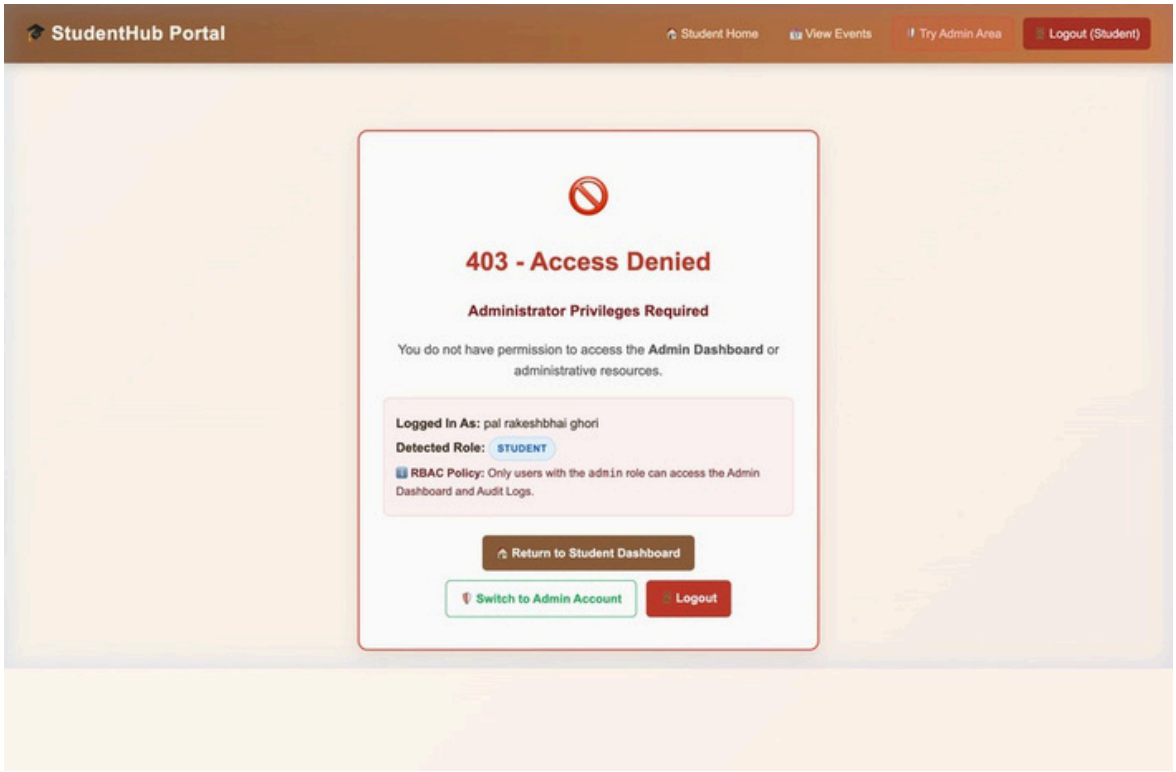
Procedural Architecture: Built with beginner-accessible procedural PHP (session_start(), mysqli_query(), mysqli_fetch_assoc()).
RBAC: Enforced server-side using $_SESSION['role'] checks on all protected routes.
Dynamic Menu: Reusable navbar.php component utilizing basic if/elseif conditions to render menu options according to role.
Audit System: Centralized record_audit_log() helper inserting structured event trails into the database.

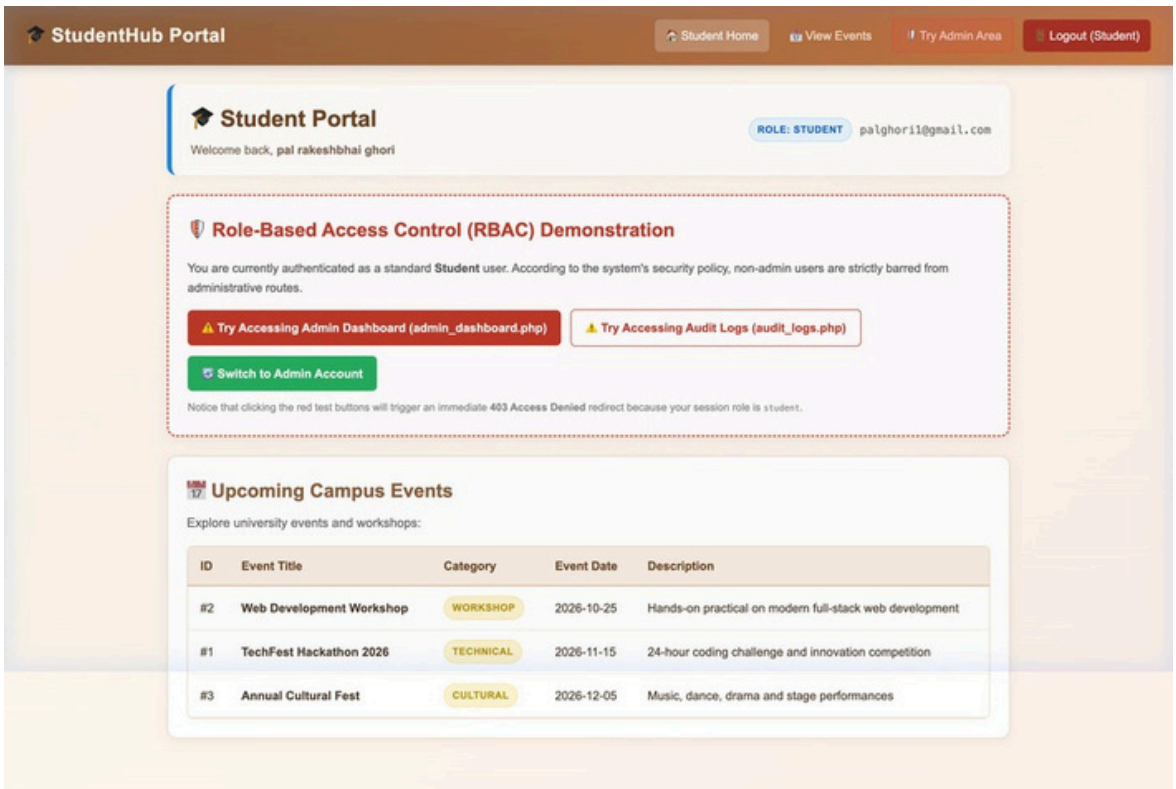
C. Evidence of Work (Screenshots)




StudentHub Portal
Dashboard
Students
Events
Audit Logs
Student View
Logout (Admin)
Administrator Dashboard
Real-Time Portal Metrics, Role-Based Access Control & Audit Trail
ROLE: ADMIN
System Admin
3
Active Students
3
Total Events
3
Registrations
1
Admin Accounts
7
Audit Log Entries
Administrative Actions (Audit Log Generator)
Executing any of these administrative actions will automatically create a timestamped record in the audit_logs table.
Type an announcement to broadcast...
Broadcast & Log
Run System Diagnostics & Log
Trigger Snapshot Backup & Log
Manage Students
Manage Events
View All Audit Logs
Latest Registrations
Loaded dynamically via SQL JOIN
Reg ID
Student Name
Student Email
Event Title
Registration Date
#3
Aarav Sharma
aarav@studenthub.com
WEB DEVELOPMENT WORKSHOP
2026-10-13 21:34:49
#2
System Admin
admin@studenthub.com
TECHFEST HACKATHON 2026
2026-10-13 21:34:49
#1
pal rakeshbhai ghori
palghori1@gmail.com
TECHFEST HACKATHON 2026
2026-10-13 21:34:49
Recent Audit Log Trail
View All Logs →
Log ID
Action
User Email
Role
Details
IP Address
Times
#7
DEMO_LOGIN
admin@studenthub.com
ADMIN
Logged in via Quick Demo as admin (admin@studenthub.com)
::1
#6
DEMO_LOGIN
palghori1@gmail.com
STUDENT
Quick demo login as student (palghori1@gmail.com)
::1
#5
DEMO_LOGIN
admin@studenthub.com
ADMIN
Quick demo login as admin (admin@studenthub.com)
::1
#4
DEMO_LOGIN
palghori1@gmail.com
STUDENT
Logged in via Quick Demo as student (palghori1@gmail.com)
::1
#3
BROADCAST_ANNOUNCEMENT
admin@studenthub.com
ADMIN
Admin posted announcement: Semester Examinations Schedule Released
::1
#2
DEMO_LOGIN
admin@studenthub.com
ADMIN
Logged in via Quick Demo as admin (admin@studenthub.com)
::1
StudentHub Portal © 2026 | Practical 14: Role-Based Access Control, Dynamic Menu & Audit Logs


StudentHub Portal
Dashboard
Students
Events
Audit Logs
Student View
Logout (Admin)
Security Audit Log Trail
Immutable record of all authentication, administrative and system events
← Back to Dashboard
Filter by Action:
-- All Actions --
Clear All Logs
ID
Action
User Email
Role
Details
IP Address
Times
#7
DEMO_LOGIN
admin@studenthub.com
ADMIN
Logged in via Quick Demo as admin (admin@studenthub.com)
::1
#6
DEMO_LOGIN
palghori1@gmail.com
STUDENT
Quick demo login as student (palghori1@gmail.com)
::1
#5
DEMO_LOGIN
admin@studenthub.com
ADMIN
Quick demo login as admin (admin@studenthub.com)
::1
#4
DEMO_LOGIN
palghori1@gmail.com
STUDENT
Logged in via Quick Demo as student (palghori1@gmail.com)
::1
#3
BROADCAST_ANNOUNCEMENT
admin@studenthub.com
ADMIN
Admin posted announcement: Semester Examinations Schedule Released
::1
#2
DEMO_LOGIN
admin@studenthub.com
ADMIN
Logged in via Quick Demo as admin (admin@studenthub.com)
::1
#1
SYSTEM_INIT
system@studenthub.com
SYSTEM
Practical 14 Database initialized with RBAC and Audit Logging
127.0.0.1
StudentHub Portal © 2026 | Practical 14: Audit Log Trail


StudentHub Portal
Student Home
View Events
Try Admin Area
Logout (Student)
403 - Access Denied
Administrator Privileges Required
You do not have permission to access the Admin Dashboard or administrative resources.
Logged In As: pal rakeshbhai ghori
Detected Role: STUDENT
RBAC Policy: Only users with the admin role can access the Admin Dashboard and Audit Logs.
Return to Student Dashboard
Switch to Admin Account
Logout


StudentHub Portal
Student Home
View Events
Try Admin Area
Logout (Student)
Student Portal
Welcome back, pal rakeshbhai ghori
ROLE: STUDENT
palghori1@gmail.com
Role-Based Access Control (RBAC) Demonstration
You are currently authenticated as a standard Student user. According to the system's security policy, non-admin users are strictly barred from administrative routes.
Try Accessing Admin Dashboard (admin_dashboard.php)
Try Accessing Audit Logs (audit_logs.php)
Switch to Admin Account
Notice that clicking the red test buttons will trigger an immediate 403 Access Denied redirect because your session role is student.
Upcoming Campus Events
Explore university events and workshops:
ID
Event Title
Category
Event Date
Description
#2
Web Development Workshop
WORKSHOP
2026-10-25
Hands-on practical on modern full-stack web development
#1
TechFest Hackathon 2026
TECHNICAL
2026-11-15
24-hour coding challenge and innovation competition
#3
Annual Cultural Fest
CULTURAL
2026-12-05
Music, dance, drama and stage performances

D. GitHub Repository Link:

https://github.com/palghori/STUDENT-HUB-PORTAL-WDF/tree/main/practical%214

E. Challenges Faced:

1. Session State Synchronization: Ensuring session destruction on logout completely resets both the PHP session array and browser cookie before redirecting.
2. Access Control Without Code Duplication: Creating a centralized access_denied.php handler and reusable navbar.php so role logic remains clean without repeating code across multiple pages.
3. Live Action Tracking: Ensuring audit logging occurs immediately alongside database operations (like INSERT/DELETE) without complicating the query flow.

F. Learning Outcomes:

1. Understood the fundamentals of Role-Based Access Control (RBAC) and server-side authorization beyond basic authentication.
2. Learned how to write SQL aggregate queries (COUNT(*)) and multi-table JOIN statements to feed real-time dashboards.
3. Learned how to implement Dynamic UI Navigation in PHP based on session variables.
4. Understood the importance of Audit Trails in web security for monitoring user activities and detecting unauthorized access attempts.

Faculty Remarks:

Practical - 15

Final Integration, Testing, Documentation, GitHub Submission, and Local Deployment

Problem Definition : Integrate all modules into a single deployable StudentHub portal. Verify navigation, responsiveness, authentication, database operations, security checks, AJAX functionality, and documentation. Deploy locally using XAMPP/Apache virtual host and submit GitHub repository.

A. Theory

1. Are all pages properly linked and navigable?

Yes. All 15+ pages are linked seamlessly via a unified navbar.php component. Navigation dynamically shows role-appropriate links (Guest, Student, Admin) and active link highlighting, with breadcrumb buttons to return to dashboards from every sub-page.

2. Are sessions, roles, validations, and database operations working end-to-end?

Yes.

- Sessions & Roles: Users are authenticated with password_verify(), session IDs are regenerated (session_regenerate_id(true)), and roles (admin/student) dictate route permissions.
- Validations: Form validations, duplicate checks (email/enrollment), and file validation (types JPG/PNG/WEBP, size ≤ 2MB) work on both frontend and backend.
- Database CRUD: SELECT, INSERT, UPDATE, and DELETE operations function end-to-end across students, events, registrations, and audit logs.

3. Are security and accessibility checklists completed?

Yes.

- Security: SQL Injection prevention (mysqli_real_escape_string), XSS sanitization (htmlspecialchars), Bcrypt password hashing (password_hash), server-side RBAC route guards, file MIME validation, and persistent audit logging.
- Accessibility: Semantic HTML5 elements (<nav>, <main>, <section>), visible :focus outlines for keyboard navigation, high-contrast readable color schemes, and descriptive alt tags on all poster thumbnails.

4. Is project documentation sufficient for installation and testing?

Yes. The README.md and TESTING_CHECKLIST.md provide complete instructions, including XAMPP installation, database import, Apache VirtualHost configuration (studenthub.local), default credentials, and test cases.

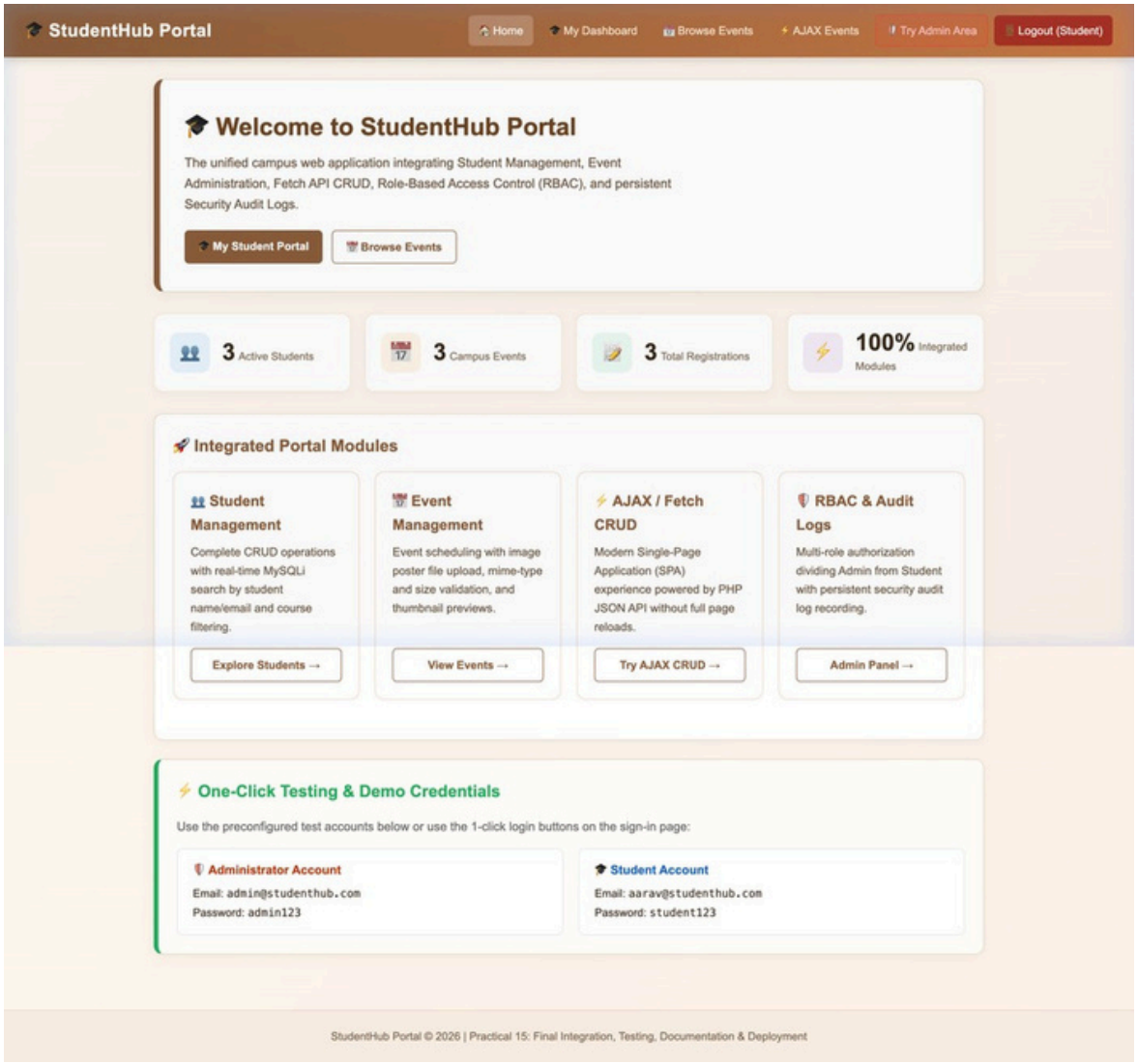
5. Is the project repository clean and properly committed?

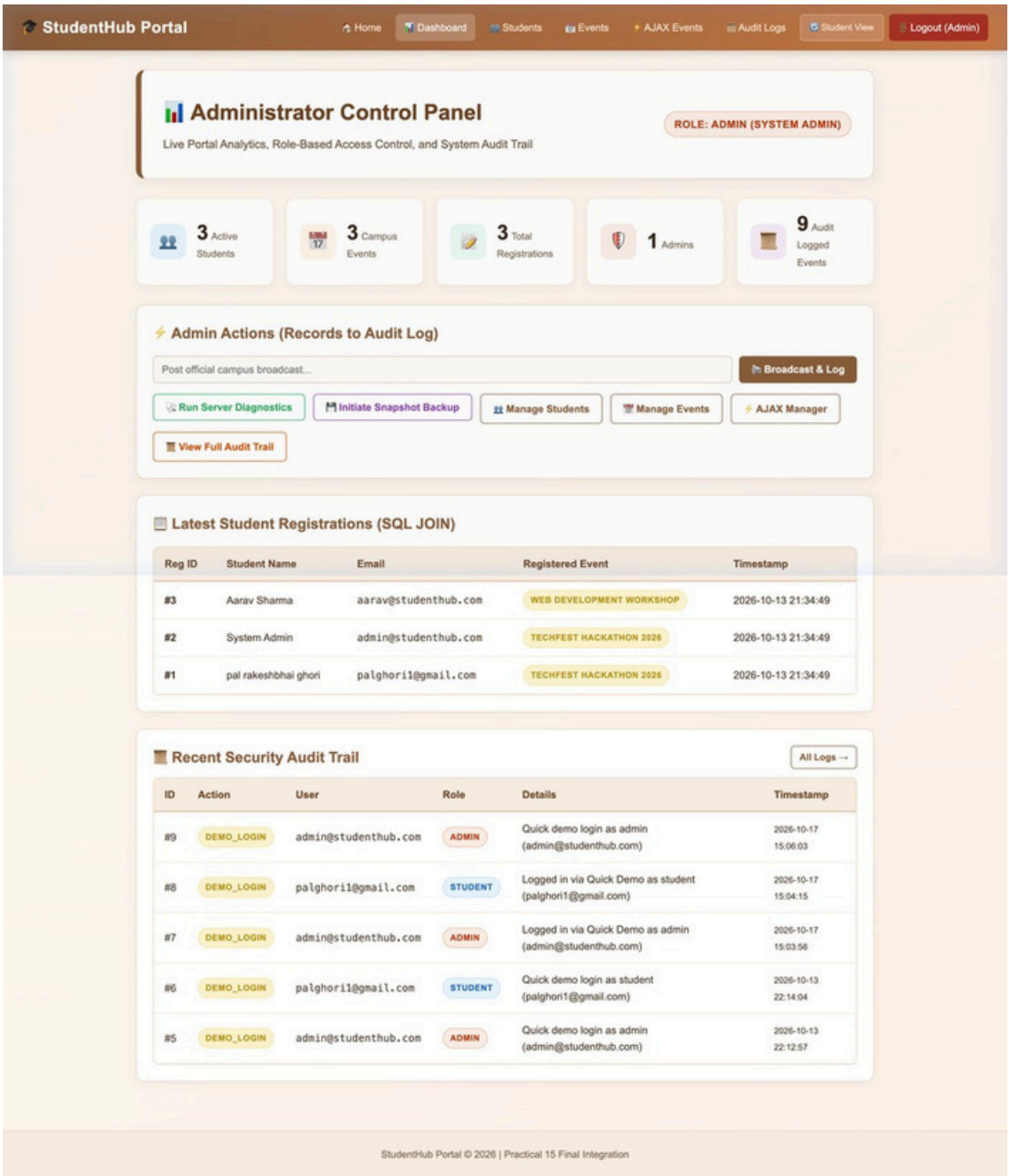
Yes. A .gitignore file was created to ignore .DS_Store and temporary files. All practical files (12, 13, 14, and 15) were staged and cleanly committed to the local main branch with a descriptive commit message: [main 1c05441] feat: complete practical 12, 13, 14, and 15 final integration, testing, and deployment documentation The working tree is completely clean (nothing to commit, working tree clean).

B. Implementation

- Consolidated all separate components (registration, session login, student CRUD, event management with file uploads, JSON API with Fetch API, and role-based admin dashboard) into a unified codebase.
- Standardized file paths, database connections, and session handling using procedural PHP and MySQLi.
- Created deployment assets including vhost_example.conf for Apache virtual hosting and studenthub_final.sql for 1-click database initialization.

C. Evidence of Work (Screenshots)




StudentHub Portal
Home
Dashboard
Students
Events
AJAX Events
Audit Logs
Student View
Logout (Admin)
Administrator Control Panel
Live Portal Analytics, Role-Based Access Control, and System Audit Trail
ROLE: ADMIN (SYSTEM ADMIN)
3 Active Students
3 Campus Events
3 Total Registrations
1 Admins
9 Audit Logged Events
Admin Actions (Records to Audit Log)
Post official campus broadcast...
Broadcast & Log
Run Server Diagnostics
Initiate Snapshot Backup
Manage Students
Manage Events
AJAX Manager
View Full Audit Trail
Latest Student Registrations (SQL JOIN)
Reg ID
Student Name
Email
Registered Event
Timestamp
#3
Aarav Sharma
aarav@studenthub.com
WEB DEVELOPMENT WORKSHOP
2026-10-13 21:34:49
#2
System Admin
admin@studenthub.com
TECHFEST HACKATHON 2026
2026-10-13 21:34:49
#1
pal rakeshbhai ghori
palghori1@gmail.com
TECHFEST HACKATHON 2026
2026-10-13 21:34:49
Recent Security Audit Trail
All Logs →
ID
Action
User
Role
Details
Timestamp
#9
DEMO_LOGIN
admin@studenthub.com
ADMIN
Quick demo login as admin (admin@studenthub.com)
2026-10-17 15:06:03
#8
DEMO_LOGIN
palghori1@gmail.com
STUDENT
Logged in via Quick Demo as student (palghori1@gmail.com)
2026-10-17 15:04:15
#7
DEMO_LOGIN
admin@studenthub.com
ADMIN
Logged in via Quick Demo as admin (admin@studenthub.com)
2026-10-17 15:03:56
#6
DEMO_LOGIN
palghori1@gmail.com
STUDENT
Quick demo login as student (palghori1@gmail.com)
2026-10-13 22:14:04
#5
DEMO_LOGIN
admin@studenthub.com
ADMIN
Quick demo login as admin (admin@studenthub.com)
2026-10-13 22:12:57
StudentHub Portal © 2026 | Practical 15 Final Integration

D. GitHub Repository Link:

https://github.com/palghori/STUDENT-HUB-PORTAL-WDF/tree/main/practical%215

E. Challenges Faced:

1.Path and Asset Consistency: Integrating standalone practicals required harmonizing relative paths for stylesheets, uploaded posters, and database helpers across all pages.
2.Role Isolation: Ensuring that student users cannot bypass navigation restrictions by manually entering administrative URLs in the address bar.
3.Asynchronous & Synchronous Coexistence: Making sure the traditional PHP CRUD pages and the modern Fetch API single-page interface share the exact same MySQL tables seamlessly without data divergence.

F. Learning Outcomes:

1.Mastered full-stack web integration by uniting discrete frontend, backend, database, and API layers into a cohesive portal.
2.Learned how to configure local Apache Virtual Hosts and manage custom domain routing via the system hosts file.
3.Gained hands-on experience in writing enterprise-grade project documentation, QA verification checklists, and maintaining clean Git version control workflows.

Faculty Remarks: